

Flutter Data Persistence

SQLite • Preferences • Files

Eng | [Ruqaih Hussein Salman](#)

2027-2028

Table of Contents

Identity-preserving engineering structure

Section	Coverage
1	Persistence Mental Model - lifetimes, serialization, async UI
2	Choosing the Right Storage Tool - selection, platform, security
3	SQLite with sqflite - official Dog/Fido + original Sqflite Roq App + migrations
4	Preferences with shared_preferences - original screens with modern Async API
5	File Storage - official counter recipe + original Textfile Roq App + JSON extension
6	Integrated Architecture and Offline-First Thinking
7	Debugging and Testing
8	Guided Practice Lab
9	Knowledge Check and Assessment
10	Answer Key, Quick Reference, and Official Sources

Learning Outcomes and Study Map

What the student should explain, choose, implement, verify, and diagnose

Area	Student mastery
Mental model	Separate widget state, process memory, persistent local storage, cache, and remote data.
Selection	Choose preferences, SQLite, or files from data shape, query needs, platform, lifetime, and risk.
SQLite	Implement the official Dog/Fido CRUD example, then apply CRUD to the original Sqflite Roq App.
Preferences	Use the original Save/Retrieve/Delete and restart screens while applying the current SharedPreferencesAsync API.
Files	Use the current counter recipe and the original Textfile Roq App, then extend to validated JSON.
Architecture	Keep storage details behind services/helpers and make UI states explicit.
Quality	Diagnose persistence failures, test restart restoration, and avoid treating ordinary local storage as secure storage.

IDENTITY RULE • Examples, visible app titles, and original screenshots are retained when still technically valid. APIs and lifecycle handling are modernized underneath them.

Visual Roadmap

Model → select → implement → verify → design, without replacing the original teaching examples

LECTURE 9 LEARNING PATH

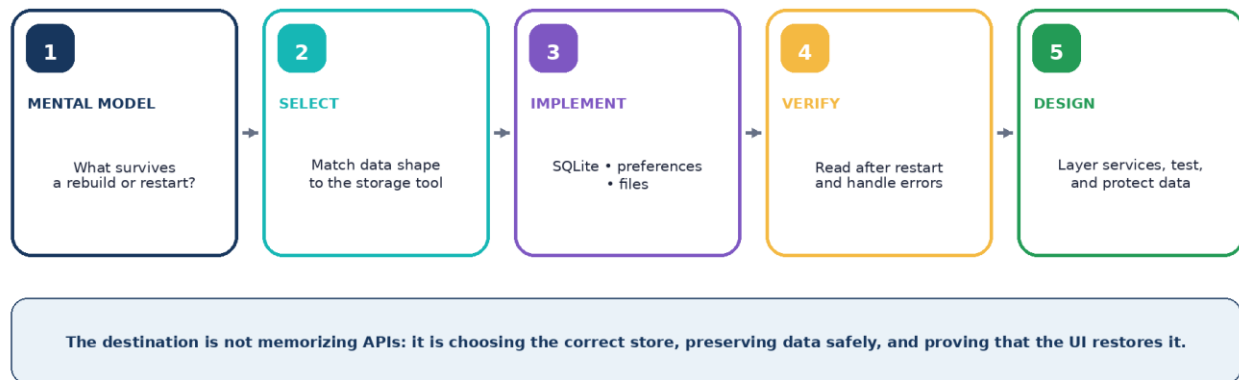


Figure 1. Five-stage learning path retained from the engineered 2027–2028 structure.

Each technical section follows the same rhythm: define the storage contract, prove it with a small focused example, connect it to the original Flutter UI, then verify restart, empty, and error behavior.

MASTERY TARGET • A write is not proof of persistence. The stronger proof is: save → terminate the process → relaunch → read → render the restored value.

Visual Evidence Legend

The lecture now distinguishes three visual roles so students know what is historical evidence, what is a teaching reconstruction, and what is a modern explanatory graphic.

Visual label	Meaning	How it is used
ORIGINAL EMULATOR CAPTURE	A screenshot that existed in the 2025–2026 lecture.	Preserve it unchanged as the identity/reference output of the original example.
RECONSTRUCTED INSTRUCTIONAL SCREEN	A screen recreated from the original code and visual style where the old lecture did not capture that operation.	Use it to explain Update/Delete steps, but do not present it as an original historical screenshot.
MODERN TEACHING INFOGRAPHIC	A designed visual summary of workflow, architecture, or verification.	Use it as an annotated teaching plate, not as runtime evidence.
Academic integrity rule • Do not label a reconstructed or generated visual as “Output”. Runtime evidence should come from a real emulator/device execution; reconstructed visuals are explicitly labeled as instructional.		

Standard Practical Example Rhythm

Every major storage mechanism follows the same teaching rhythm so students first prove the storage boundary, then connect it to a real Flutter interface.

Stage	What students do	Evidence
1 • Understand	Identify the data lifetime, shape, and operation.	A short decision statement.
2 • Prepare	Add packages, identify files, and explain where each code block belongs.	Project/file map + package command.
3 • Console-first	Run one complete Flutter entry point with no Material UI.	Ordered Debug Console output.
4 • Read the output	Explain what each printed line proves about the real store.	Create/Read/Update/Delete or Save/Read/Reset evidence.
5 • Flutter UI	Connect the same storage contract to the original application screens.	Original captures + clearly labeled reconstructed states.
6 • Restart/Failure proof	Terminate/relaunch and test absent/error paths.	Restored value or intentional empty/error state.
7 • Engineering extension	Add current API, lifecycle, migration, architecture, testing, and security guidance.	Modern improvement without replacing the original example.
WHY THIS RHYTHM • The console proof isolates persistence from UI state. The Flutter application then proves that the visible interface is driven by the same stored data. This avoids teaching students to confuse <code>setState()</code> with persistence.		

1. Persistence Mental Model

What survives, where it lives, and how the UI gets it back

Flutter widgets rebuild frequently, but persistent data lives outside widget memory. A later app process must be able to read the stored representation and reconstruct the intended UI state.

THE PERSISTENCE PIPELINE

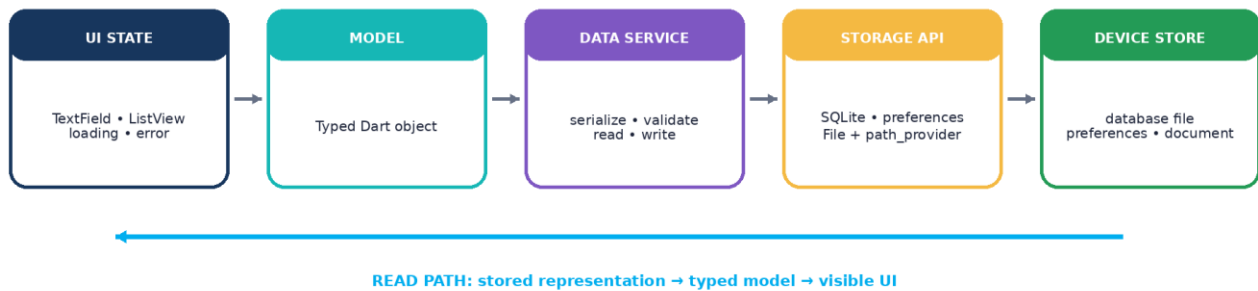


Figure 2. Persistence pipeline: UI state → model → data service → storage API → device store.

1.1. Four Lifetimes Students Must Separate

Lifetime	What changes	Engineering consequence
Widget rebuild	Local variables inside build() are recreated.	Use fields/state management for current UI state.
State object lifetime	Fields survive rebuilds while State remains mounted.	dispose() ends owned controllers/subscriptions.
App process	Memory disappears when the process is killed.	Persist values that must be restored later.
App installation	App-private data commonly survives restart but is normally removed on uninstall.	Treat backup/restore separately.
Temporary cache	The OS may clear it when space is needed.	Never make cache the only copy of required user data.

1.2. Serialization Is the Boundary

Objects must become representations understood by the storage API

Dart value	Stored representation	Restore step
DateTime	ISO-8601 TEXT or integer epoch	DateTime.parse() or fromMillisecondsSinceEpoch()
bool in SQLite	INTEGER 0 / 1	value == 1
Model in JSON	Map<String, Object?> → JSON string	jsonDecode() → validated factory
Nullable value	NULL / missing key / absent file	Intentional default or empty state

NULL IS A VALID OUTCOME • A missing preference key, zero database rows, or a missing file often represents first run. Design that state rather than treating every absence as a fault.

1.3. Persistence Work Is Asynchronous

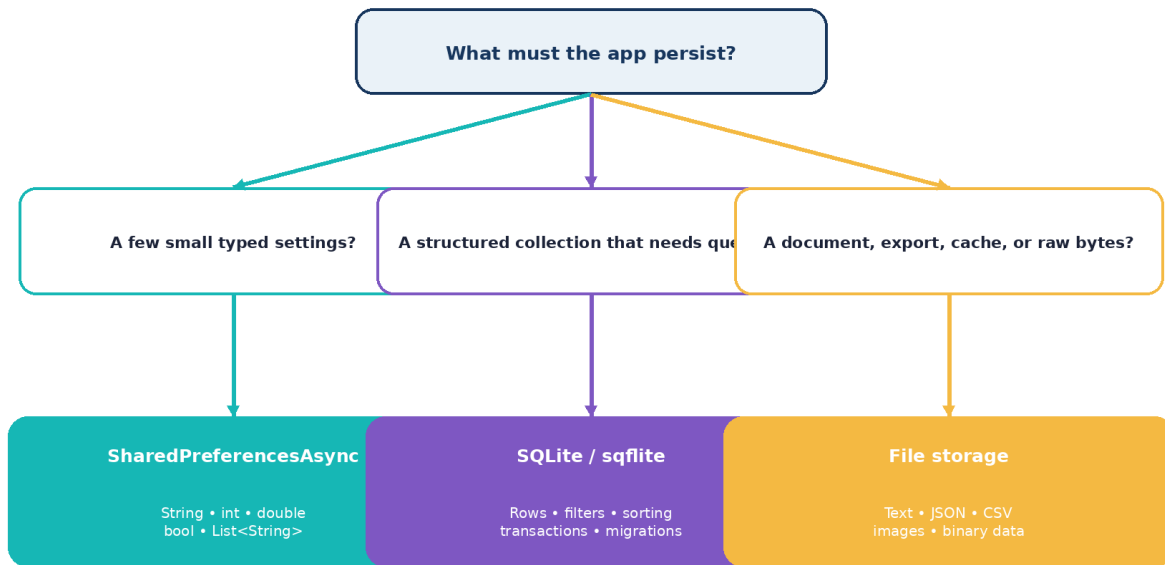
- Start a read once - for example in initState(), a view model, or a repository - not inside every build() call.
- Render explicit loading, empty/data, and error states.
- After await inside State, check mounted before setState(), Navigator, or ScaffoldMessenger access.
- Await writes when later behavior depends on success; show success only after the write completes.

KEY DISTINCTION • setState() changes the current interface; a storage write changes persistent representation. They are related but can succeed or fail independently.

2. Choosing the Right Storage Tool

Decide from the data contract before writing package code

STORAGE DECISION MAP



If the data is secret or safety-critical, add an appropriate secure-storage/encryption design; none of these choices is automatically secure.

Figure 3. Storage selection begins with data shape and operations, then checks platform and risk.

2.1. Core Decision Matrix

Mechanism	Best data shape	Typical use	Primary caution
SharedPreferencesAsync	A few small typed settings	Name, flags, display settings	Not for large/complex or critical data
SQLite / sqflite	Structured collections and relationships	Journals, notes, inventory, offline records	Schema and migration responsibility
File storage	Documents, exports, cache, raw bytes	TXT/JSON/CSV, export, downloaded content	App owns format, parsing, and corruption handling

2.2. Platform Reality

Tool	Direct package support / recipe	Design note
shared_preferences	Android, iOS, Linux, macOS, Web, Windows	New work should prefer Async or WithCache APIs.
sqflite	Android, iOS, macOS	Linux/Windows can use sqflite_common_ffi; web is a different/experimental path.
path_provider + dart:io	Mobile and desktop recipe	The standard Flutter cookbook read/write-files recipe does not work as written for web.

SECURITY BOUNDARY • Preferences, ordinary SQLite databases, and normal files are not automatically encrypted. Do not teach password/token storage in them as a default pattern.

2.3. Fast Selection Algorithm

1. Define the value lifetime: screen, session, across restarts, or synchronized across devices.
2. Describe the shape: one typed value, many structured records, or a document/binary payload.
3. List operations: direct lookup, filtering/sorting, transaction, export/import, streaming.
4. Check the actual target platforms and package support.
5. Classify risk: disposable cache, normal user data, critical state, or secret material.
6. Choose one authoritative home for each datum and expose it through a helper/service/repository.

3. SQLite with sqflite

Keep the original Dog/Fido foundation and Sqflite Roq App; modernize the implementation

The original SQLite identity is preserved because the Dog/Fido CRUD recipe is still the official Flutter cookbook teaching example. The original orange Sqflite Roq App remains the UI example. Engineering additions are layered around those examples rather than replacing them.



Powered by **NeoLogic**

SQL Database Storage Using Sqlite In Flutter



Original lecture visual retained: SQL Database Storage Using Sqflite in Flutter.

3.1. Project Setup and Platform Note

TERMINAL • Add current compatible packages; do not freeze tutorial-era versions

```
flutter pub add sqflite path
```

- Call `WidgetsFlutterBinding.ensureInitialized()` before plugin use from an async entry point.
- Build the database path with `path.join(await getDatabasesPath(), fileName)`.
- Use a schema version and `onUpgrade` when a released schema changes.
- `sqflite` directly targets Android, iOS, and macOS; other platforms require a separate path.

3.2. Official Foundation Example: Dog / Fido

The original example stays; the presentation is modernized into short visual steps before each code block.

Read each step in the same rhythm: understand the operation in seconds, inspect the key API, then type the code. The full console program and its output come after the individual operations.

3.2.1. Define the Dog model

Describe one stored record before touching SQLite.

PURPOSE

Represent one dog with id, name, and age.

KEY API

Dart class + `toMap()`

CHECKPOINT

A typed object can be converted into SQLite column values.

DART • dog.dart

```
class Dog {
  const Dog({required this.id, required this.name, required this.age});

  final int id;
  final String name;
  final int age;

  Map<String, Object?> toMap() => <String, Object?>{
    'id': id,
    'name': name,
    'age': age,
  };

  @override
  String toString() => 'Dog{id: $id, name: $name, age: $age}';
}
```

3.2.2. Open the database*Locate the database file, then open one connection.***PURPOSE**

Prepare the database before any read or write.

KEY API

getDatabasesPath() → join() → openDatabase()

CHECKPOINT

doggie_database.db is opened or created at the platform database path.

DART • Open the database

```
WidgetsFlutterBinding.ensureInitialized();

final database = openDatabase(
  join(await getDatabasesPath(), 'doggie_database.db'),
);
```

WHY join()? • It builds the path correctly for the current platform instead of hard-coding path separators.

3.2.3. Create the dogs table*Define the schema only when the database is first created.***PURPOSE**

Create columns that match the Dog model.

KEY API

onCreate + db.execute() + version

CHECKPOINT

The database contains dogs(id, name, age).

DART • Add the schema to openDatabase()

```
final database = openDatabase(
  join(await getDatabasesPath(), 'doggie_database.db'),
  onCreate: (db, version) {
    return db.execute(
      'CREATE TABLE dogs(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)',
    );
  },
  version: 1,
);
```

SCHEMA NOTE • Changing CREATE TABLE later does not modify an already-installed database. A released schema change needs a higher version and onUpgrade.

3.2.4. Insert a Dog*Convert the object to column values and write one row.***PURPOSE**

Store a Dog in the dogs table.

KEY API

db.insert() + dog.toMap() + await

CHECKPOINT

The row is persisted before the next operation runs.

DART • Insert

```
Future<void> insertDog(Dog dog) async {
  final db = await database;
  await db.insert(
    'dogs',
    dog.toMap(),
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}
```

3.2.5. Retrieve the Dogs

Read rows from SQLite and rebuild typed Dog objects.

PURPOSE

Verify what the database actually stored.

KEY API

db.query() → Map rows → Dog objects

CHECKPOINT

The result is List<Dog>, not raw database rows.

DART • Query / Read

```
Future<List<Dog>> dogs() async {
  final db = await database;
  final maps = await db.query('dogs');

  return maps
    .map((map) => Dog(
      id: map['id'] as int,
      name: map['name'] as String,
      age: map['age'] as int,
    ))
    .toList();
}
```

READ-BACK RULE • A fresh query is stronger evidence than printing the Dog object already held in memory.

3.2.6. Update one Dog

Write changed values while targeting exactly one row.

PURPOSE

Change Fido without affecting other records.

KEY API

db.update() + where: 'id = ?' + whereArgs

CHECKPOINT

A new query returns the updated age for the same id.

DART • Update

```
Future<void> updateDog(Dog dog) async {
  final db = await database;
  await db.update(
    'dogs',
    dog.toMap(),
    where: 'id = ?',
    whereArgs: <Object?>[dog.id],
  );
}
```

SAFE TARGETING • Use placeholders and whereArgs for values instead of interpolating external values into SQL.

3.2.7. Delete one Dog

Remove the row by id, then verify it is gone.

PURPOSE

Delete only the selected Dog.

KEY API

db.delete() + where: 'id = ?' + whereArgs

CHECKPOINT

The final query no longer returns that row.

DART • Delete

```
Future<void> deleteDog(int id) async {
  final db = await database;
  await db.delete(
    'dogs',
    where: 'id = ?',
    whereArgs: <Object?>[id],
  );
}
```

3.2.8. Console-First Complete Program: Fido CRUD**Create → Read → Update → Read → Delete → Verify**

Now combine the same small operations into one runnable Flutter entry point. No Material UI is involved; the Debug Console is the evidence.

DART • lib/db_test.dart - complete console proof

```
import 'package:flutter/widgets.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class Dog {
  const Dog({required this.id, required this.name, required this.age});
  final int id;
  final String name;
  final int age;

  Map<String, Object?> toMap() => {'id': id, 'name': name, 'age': age};

  @override
  String toString() => 'Dog{id: $id, name: $name, age: $age}';
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final database = openDatabase(
    join(await getDatabasesPath(), 'doggie_database.db'),
    onCreate: (db, version) => db.execute(
      'CREATE TABLE dogs(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)',
    ),
    version: 1,
  );

  Future<void> insertDog(Dog dog) async {
    final db = await database;
    await db.insert('dogs', dog.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace);
  }

  Future<List<Dog>> dogs() async {
    final db = await database;
    final maps = await db.query('dogs');
    return maps
      .map((m) => Dog(
        id: m['id'] as int,
        name: m['name'] as String,
        age: m['age'] as int,
      ))
      .toList();
  }

  Future<void> updateDog(Dog dog) async {
    final db = await database;
    await db.update('dogs', dog.toMap(),
      where: 'id = ?', whereArgs: <Object?>[dog.id]);
  }

  Future<void> deleteDog(int id) async {
    final db = await database;
    await db.delete('dogs', where: 'id = ?', whereArgs: <Object?>[id]);
  }

  var fido = const Dog(id: 0, name: 'Fido', age: 35);
  await insertDog(fido);
  print(await dogs());

  fido = Dog(id: fido.id, name: fido.name, age: 42);
```

```

await updateDog(fido);
print(await dogs());

await deleteDog(fido.id);
print(await dogs());
}

```

EXPECTED DEBUG CONSOLE OUTPUT

```

[Dog{id: 0, name: Fido, age: 35}]
[Dog{id: 0, name: Fido, age: 42}]
[]

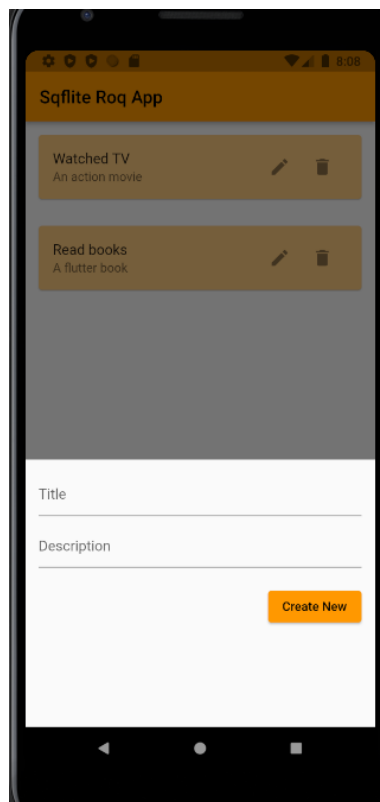
```

WHAT THE THREE LINES PROVE • 35 confirms Insert + Read. 42 confirms Update + Read. [] confirms Delete + final verification. The evidence comes from fresh database queries, not from widget memory.

NEXT: SAME CRUD, NOW WITH UI • Only after the console contract is clear do we move to the original orange Sqlite Roq App. The domain changes to journal/items, but the data boundary is the same: user action → SQLite operation → fresh query → visible screen state.

3.3. Original Flutter UI Example: Sqlite Roq App

Journal/activity CRUD - original orange screens preserved



Original app preview retained: list + bottom-sheet editor.

Column	Type	Purpose
id	INTEGER PRIMARY KEY AUTOINCREMENT	Journal identifier
title	TEXT	Activity title
description	TEXT	Activity details
createdAt	TIMESTAMP	Creation/update timestamp

DART • Sql_Helper_roq.dart - original helper, modernized safely

```

import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart' as sql;

```

```

class SQLHelper {
  static Future<void> createTables(sql.Database database) async {
    await database.execute("""
      CREATE TABLE items(
        id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
        title TEXT NOT NULL,
        description TEXT NOT NULL,
        createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
      )
    """);
  }

  static Future<sql.Database> db() async {
    final path = p.join(
      await sql.getDatabasesPath(),
      'activity_ruqaih_salman.db',
    );
    return sql.openDatabase(
      path,
      version: 1,
      onCreate: (database, version) => createTables(database),
    );
  }

  static Future<int> createItem(String title, String description) async {
    final database = await db();
    return database.insert(
      'items',
      <String, Object?>{'title': title, 'description': description},
      conflictAlgorithm: sql.ConflictAlgorithm.replace,
    );
  }

  static Future<List<Map<String, Object?>>> getItems() async {
    final database = await db();
    return database.query('items', orderBy: 'id DESC');
  }

  static Future<int> updateItem(
    int id,
    String title,
    String description,
  ) async {
    final database = await db();
    return database.update(
      'items',
      <String, Object?>{
        'title': title,
        'description': description,
        'createdAt': DateTime.now().toIso8601String(),
      },
      where: 'id = ?',
      whereArgs: <Object?>[id],
    );
  }

  static Future<int> deleteItem(int id) async {
    final database = await db();
    return database.delete(
      'items',
      where: 'id = ?',
      whereArgs: <Object?>[id],
    );
  }
}

```

DART • main.dart - preserve original UI identity, add lifecycle safety

```

class MyUi_Page_Sqlite_Roq extends StatefulWidget {
  const MyUi_Page_Sqlite_Roq({super.key});

  @override
  State<MyUi_Page_Sqlite_Roq> createState() => _HomePageState();
}

class _HomePageState extends State<MyUi_Page_Sqlite_Roq> {
  List<Map<String, Object?>> _journals = <Map<String, Object?>>[];
  bool _isLoading = true;
  Object? _error;

  final _titleController = TextEditingController();
  final _descriptionController = TextEditingController();

  @override
  void initState() {
    super.initState();
    _refreshJournals();
  }
}

```

```

}

Future<void> _refreshJournals() async {
  try {
    final data = await SQLHelper.getItems();
    if (!mounted) return;
    setState(() {
      _journals = data;
      _error = null;
    });
  } catch (error) {
    if (mounted) setState(() => _error = error);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}

@override
void dispose() {
  _titleController.dispose();
  _descriptionController.dispose();
  super.dispose();
}

// Keep the original bottom-sheet Create/Update flow and orange visual theme.
// After each awaited mutation, reload the database before showing success.
}

```

IMPLEMENTATION NOTE • The visible screen and interaction sequence remain the original lecture example. Only the underlying safety rules change: await the mutation, parameterize WHERE, check mounted, dispose controllers, and show loading/error/empty states.

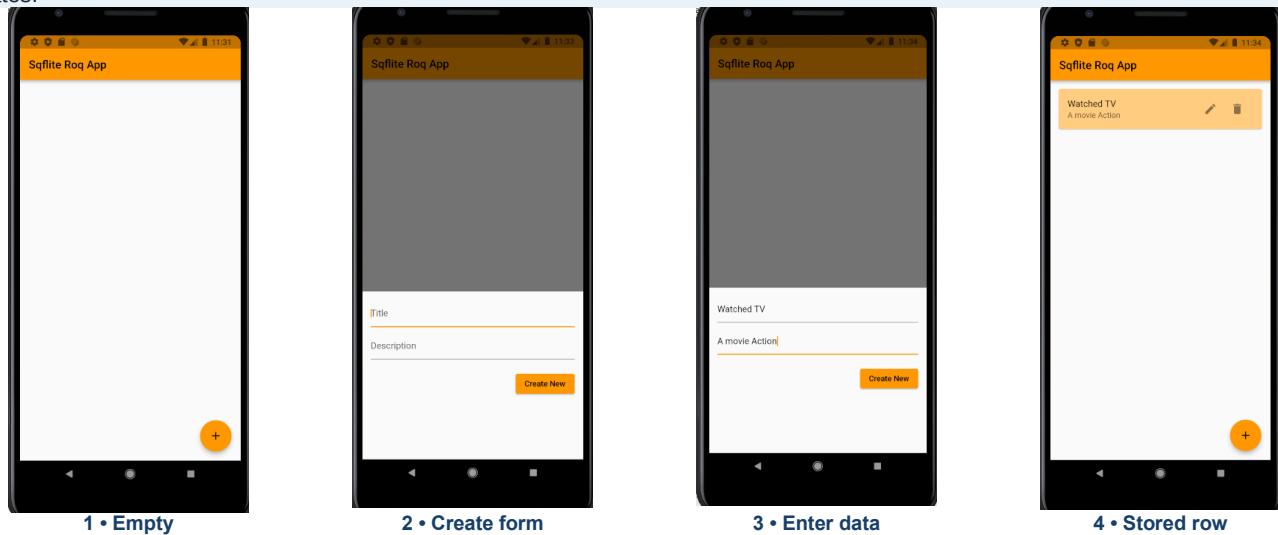


Figure 4. Original Sqflite Roq App output sequence retained from the earlier lecture.

3.3.0. Visual Teaching Plate - How to Read the Original App

Lecture 9 – Flutter Data Persistence
SQLite – sqflite Roq App (Journal / Diary)

3.2 SQLite in Flutter – sqflite Roq App (Journal / Diary)


Objective: Build a complete Flutter application (Recipe Flutter Light) that allows the user to Create, Read, Update, and Delete journal entries using SQLite through the sqflite package.

3.2.1 Application Overview

The application allows the user to manage journal entries. Each entry has:

- **title** (String)
- **description** (String)

3.2.2 User Flow Overview

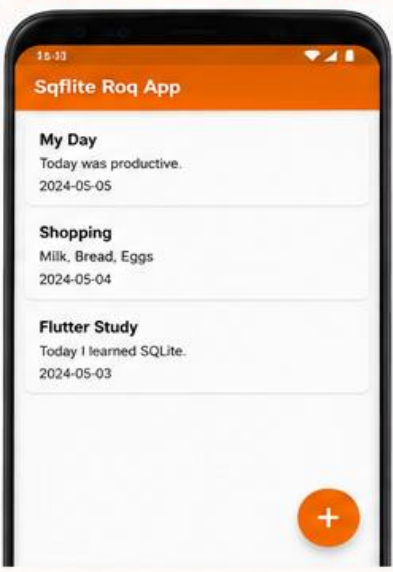


```

graph LR
    A[Open App  
Shows list of journal items] --> B[Tap + (FAB)  
Open bottom sheet to add new item]
    B --> C[Item appears in the list]
    C --> D[Tap item  
Open bottom sheet pre-filled for update]
    D --> E[Delete item  
Remove item from database and list]
  
```

3.2.3 Screens and Detailed Instructions

1 Main Screen – List of Journal Items



What you see:

- The app bar shows the app name.
- A ListView displays all saved journal items.
- Each item shows title, description and date.
- A Floating Action Button (FAB) "+" at the bottom-right to add a new item.

How it works:

- When the app starts, we query the database and load all items.
- The list is built from the in-memory list.

Instructions (Step by Step):

1. Run the app.
2. The main screen appears.
3. Tap the + (FAB) button to add a new journal item.


Note: The list is refreshed automatically after any add, update or delete operation.

MODERN TEACHING INFOGRAPHIC • A visual overview of the original Sqflite Roq App. Use this before the detailed steps to orient students; it is not an emulator capture.

Lecturer use • First show the original screenshots. Then use this infographic to name the UI elements (ListView, FloatingActionButton, bottom sheet, edit/delete controls) and map them to CRUD operations.

3.3.1. Screen-by-Screen Execution Guide

The original orange application remains the reference UI. The added screens below complete the missing Create, Update, and Delete demonstrations without replacing the original example.

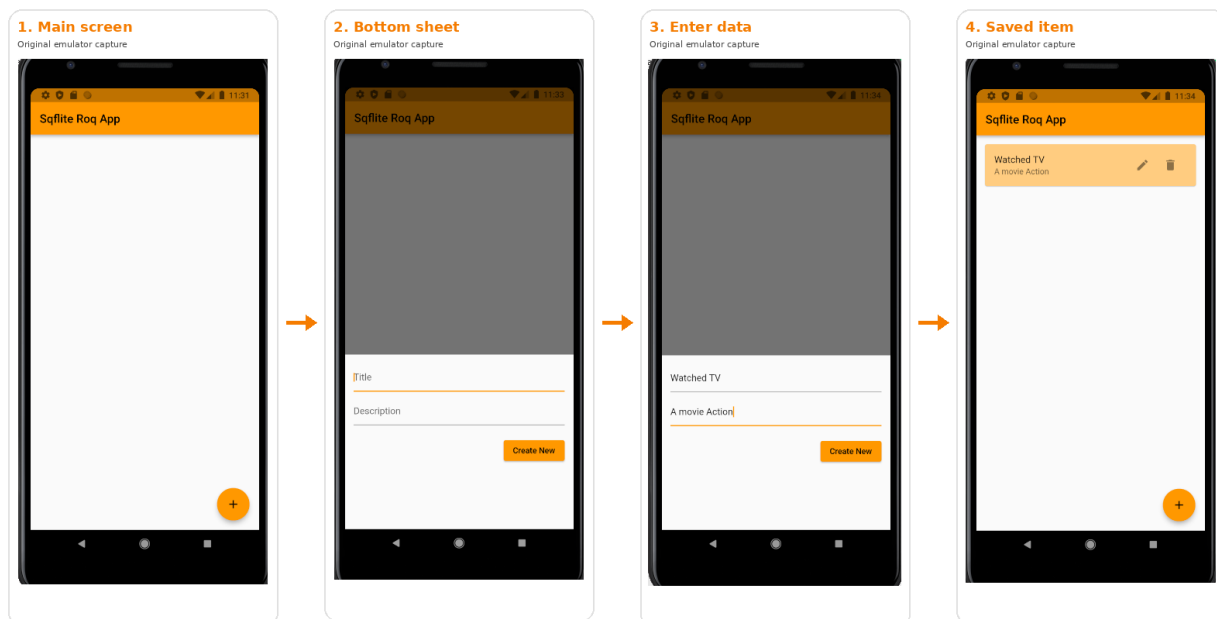
A. Main Screen - Read / List State

When the application opens, `_refreshJournals()` reads the rows from SQLite and builds the visible `ListView`. The Floating Action Button (+) at the bottom-right is the entry point for creating a new journal item.

User action	What the app does
Run the application.	<code>initState()</code> calls <code>_refreshJournals()</code> ; the database is queried and the current rows are loaded.
Inspect the journal list.	Each row shows the stored title and description. If there are no rows, show an explicit empty-state message.
Tap the + Floating Action Button.	Call <code>_showForm(null)</code> . Because <code>id</code> is null, the bottom sheet opens in Create mode with empty fields.

Teaching cue • Tell students that the + icon is a `FloatingActionButton`. Its `onPressed` callback does not save data by itself; it opens the input form. The actual database write happens only after the user confirms the form action.

B. Create - Open the Bottom Sheet and Add a New Item



ORIGINAL EMULATOR CAPTURE SEQUENCE • The four Create states are assembled only from screenshots already present in the original lecture: main screen → bottom sheet → entered values → saved row.

1. On the main screen, tap the + Floating Action Button (FAB).
2. A modal bottom sheet slides up. It contains two `TextField` controls: Title and Description.
3. Enter the new journal data. The controllers keep the temporary text in widget memory.
4. Tap ADD / Create New. Await `SQLHelper.createItem(...)` so the insert finishes before reporting success.
5. Close the bottom sheet, query the database again, and rebuild the list. The revised example orders by `id DESC` so the newest record is immediately visible.

DART • Create flow - the FAB opens Create mode

```

FloatingActionButton(
  onPressed: () => _showForm(null),
  child: const Icon(Icons.add),
)

// Inside the bottom-sheet button:
if (id == null) {
  await SQLHelper.createItem(
    _titleController.text.trim(),
    _descriptionController.text.trim(),
  )
}

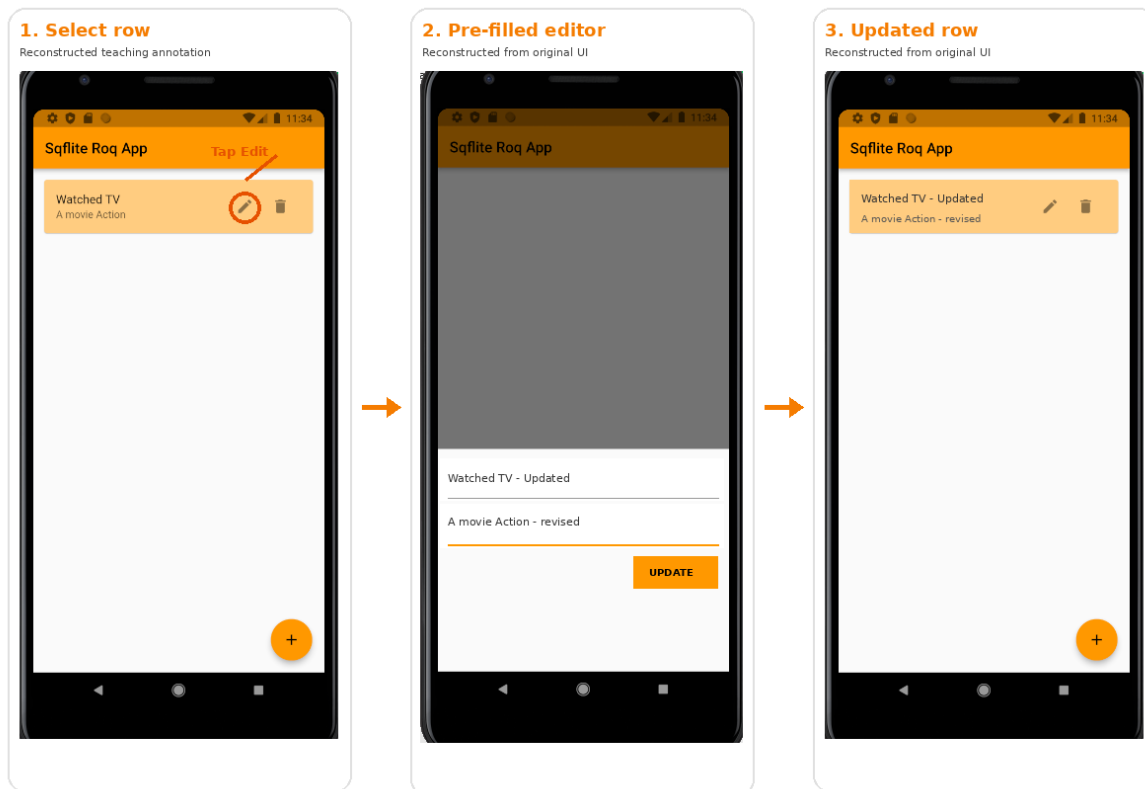
```

```

);
}
if (!mounted) return;
Navigator.of(context).pop();
await refreshJournals();

```

C. Update - Reopen the Same Bottom Sheet with Existing Data



RECONSTRUCTED INSTRUCTIONAL SCREEN • Update was not captured in the original lecture. This sequence is reconstructed from the original orange UI and the original update code: select row → pre-filled editor → UPDATE → refreshed row.

1. Tap the edit icon (pencil) on the journal row you want to change.
2. Pass that row id to `_showForm(id)`. Because id is not null, the current title and description are copied into the `TextEditingController`s.
3. The same bottom sheet now acts as an Edit form; students can see that Create and Update share one UI but use different database operations.
4. Change one or both fields and tap UPDATE.
5. Await `SQLHelper.updateItem(id, ...)`. The SQL helper targets exactly one row with where: 'id = ?' and whereArgs: [id].
6. Close the sheet, reload from SQLite, and verify the changed value from the database rather than only from local widget memory.

DART • Update flow - pre-fill and target one row safely

```

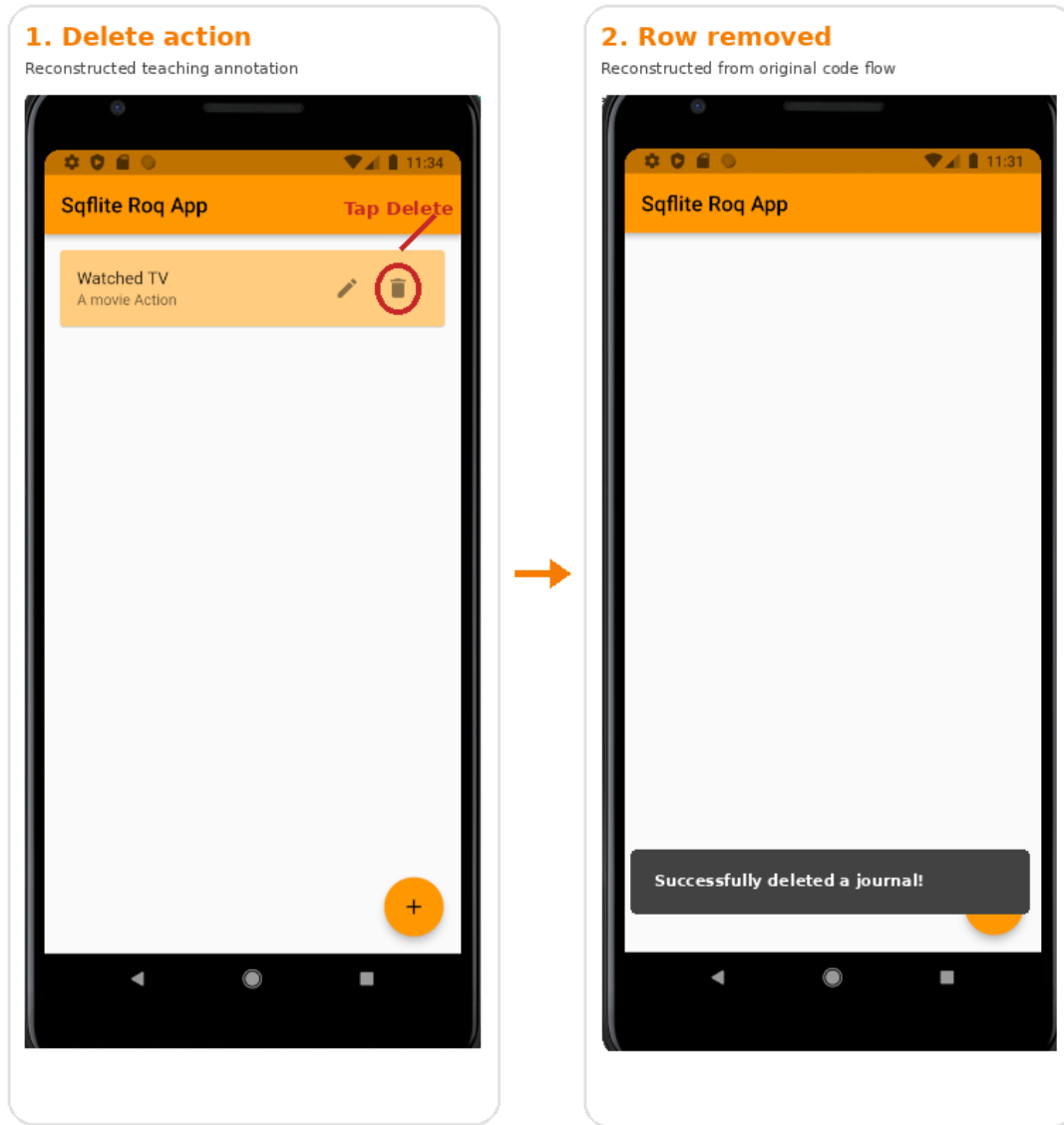
if (id != null) {
  final existing = _journals.firstWhere(
    (item) => item['id'] == id,
  );
  _titleController.text = existing['title'] as String;
  _descriptionController.text = existing['description'] as String;
}

await SQLHelper.updateItem(
  id,
  _titleController.text.trim(),
  _descriptionController.text.trim(),
);
await refreshJournals();

```

Engineering checkpoint • Update should be proven by a new database read. Do not teach students to modify only the in-memory List and assume persistence succeeded.

D. Delete - Remove a Row and Verify the Result



RECONSTRUCTED INSTRUCTIONAL SCREEN • Delete was not captured in the original lecture. This sequence follows the original code behavior: tap trash → delete row → refresh list → show success Snackbar.

1. Tap the trash/delete icon beside the target journal item.
2. Original code behavior: the delete icon calls `SQLHelper.deleteItem(id)` directly. The row is removed, a `SnackBar` reports success, and the list is refreshed.
3. Call `SQLHelper.deleteItem(id)` and await the `Future`. The original example deletes directly; confirmation is introduced later as an optional UX extension.
4. The helper uses where: `'id = ?'` with `whereArgs` so only the intended row is targeted.
5. Reload the database. The deleted row must no longer appear in the `ListView`.
6. For classroom verification, ask students to fully restart the app and confirm that the deleted row does not return.

DART • Original delete flow - direct delete + refresh + Snackbar

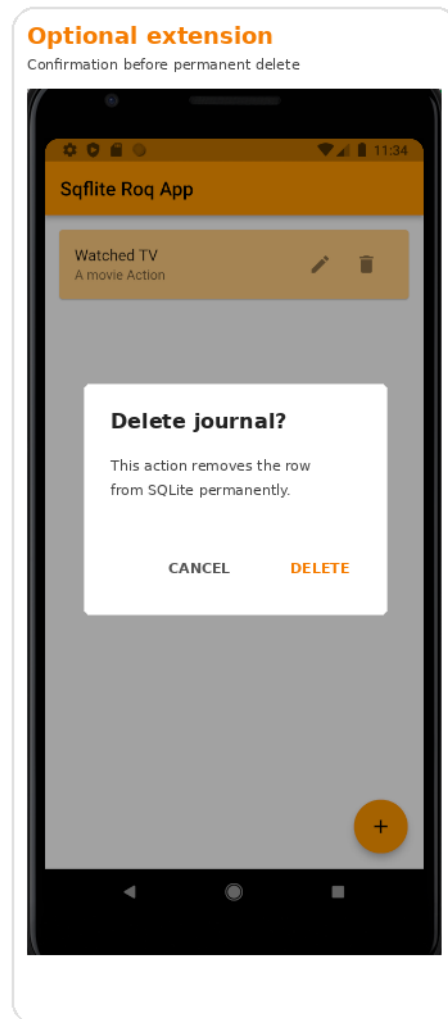
```
Future<void> _deleteItem(int id) async {
  await SQLHelper.deleteItem(id);
  await _refreshJournals();
  if (!mounted) return;

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Successfully deleted a journal!'),
    ),
  ),
}
```

```
);
}
```

Verification rule • Create, Update, and Delete are not considered demonstrated merely because the screen changed. Each operation should be followed by a database read/refresh; restart evidence is the strongest persistence check.

E. Optional UX Extension - Confirm Before Permanent Delete



MODERN TEACHING EXTENSION • Optional confirmation dialog shown in the original orange visual style. This was not part of the old application and is clearly taught as an enhancement.

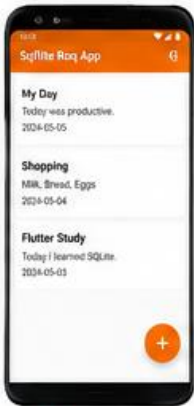
This confirmation is not required by sqflite. It is a user-experience safeguard for destructive actions. Keep the original direct-delete flow first, then show this as a small extension students can implement after CRUD works.

DART • Optional confirmation wrapper (extension, not original code)

```
final confirmed = await showDialog<bool>(
  context: context,
  builder: (context) => AlertDialog(
    title: const Text('Delete journal?'),
    content: const Text('This action permanently removes the item.'),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(context, false),
        child: const Text('Cancel'),
      ),
      FilledButton(
        onPressed: () => Navigator.pop(context, true),
        child: const Text('Delete'),
      ),
    ],
  ),
);
if (confirmed == true) {
  await SQLHelper.deleteItem(id);
  await _refreshJournals();
}
```

Lecture 9 – Flutter Data Persistence
SQLite – sqflite Roq App (Journal / Diary)

2 Add New Item – Bottom Sheet



2.1 Tap + (FAB)



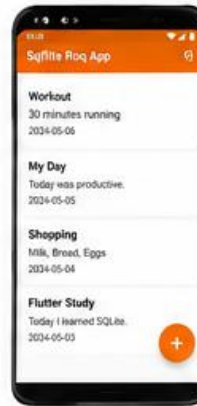
2.2 Bottom sheet opens



2.3 Enter data



2.4 Press ADD



2.5 New item appears

Detailed Instructions:

- On the main screen, tap the Floating Action Button (+).
- A modal bottom sheet slides up with two text fields: Title and Description.
- Type the title and description of your new journal item.
- Tap ADD.
- The bottom sheet closes and the new item appears at the top of the list.

Implementation Notes:

- We use `showModalBottomSheet()` to display the form.
- After pressing ADD, we insert the item into SQLite.
- Then we re-query the database and call `setState()` to refresh the list.

3 Update Item – Bottom Sheet (Pre-filled)



3.1 Tap an item



3.2 Bottom sheet opens with existing data

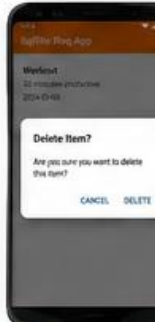


3.3 Edit and press UPDATE

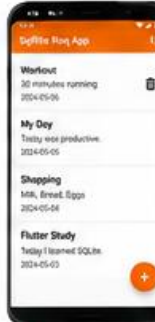
4 Delete Item – With Confirmation



4.1 Tap the delete icon (🗑️) on an item



4.2 Confirm deletion



4.3 Item is removed from the list

Detailed Instructions:

- Tap any item in the list.
- A bottom sheet opens pre-filled with the item's current data.
- Modify the title or description.
- Tap UPDATE.
- The list refreshes and shows the updated item.

Detailed Instructions:

- On the main screen, tap the delete icon (🗑️) next to the item.
- A confirmation dialog appears.
- Tap DELETE to confirm.
- The item is deleted from the database.
- The list refreshes and the item disappears.

Important:
Always confirm before permanently deleting data.

Bachelor in Information Technology (BIT)
13
Academic Year 2027 – 2028

MODERN TEACHING INFOGRAPHIC • Create, Update, and Delete workflow summarized as a teaching plate. Pair it with the real/original captures and the clearly labeled reconstructed screens.

3.3.2. Current Flutter UI Note - Material 3 and Screenshot Fidelity

The original orange screenshots were captured with an older Material visual system. In current Flutter, Material 3 is the default, so the same functional code can render buttons, FloatingActionButton shapes, spacing, typography, and colors differently. That visual drift does not mean the persistence logic is wrong.

Goal	Recommended treatment
Preserve the historical/original lecture identity	Keep the original emulator screenshots exactly as evidence. Do not redraw them and call them original output.
Reproduce the legacy look temporarily in a classroom demo	You can set <code>useMaterial3: false</code> while that compatibility switch remains available, but do not treat this as a long-term production strategy.
Teach current Flutter UI	Use Material 3 with a <code>ColorScheme</code> , and explain that the visuals are current while CRUD/storage behavior remains the same.
DART • Two visual modes for teaching the same persistence logic	
<pre>// A. Legacy-look demonstration only (temporary compatibility path) theme: ThemeData(useMaterial3: false, primarySwatch: Colors.orange,), // B. Current Material 3 direction theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: Colors.orange,),),</pre>	
Why the new reconstructions were not 100% identical • The missing Update/Delete images never existed as original captures, and current Flutter defaults also render Material widgets differently. The revised lecture therefore preserves originals unchanged and labels every reconstructed screen honestly.	

3.4. Engineering Upgrade: Migration and Atomic Work

DART • Version 1 → 2 migration - add pinned without discarding old data

```
Future<sql.Database> db() async {
  final path = p.join(
    await sql.getDatabasesPath(),
    'activity_ruqaih_salman.db',
  );
  return sql.openDatabase(
    path,
    version: 2,
    onCreate: (database, version) async {
      await createTables(database);
      await database.execute(
        'ALTER TABLE items ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0',
      );
    },
    onUpgrade: (database, oldVersion, newVersion) async {
      if (oldVersion < 2) {
        await database.execute(
          'ALTER TABLE items ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0',
        );
      }
    },
  );
}
```

Mechanism	Use it when	Guarantee / caution
<code>onUpgrade</code>	Installed schema must move to a newer version	Make each transition explicit and test old→new.
<code>transaction</code>	Several operations form one logical action	All succeed or the transaction rolls back.
<code>batch</code>	Many statements should be sent efficiently	Use a transaction when atomicity is required.

3.5. Current Architecture Extension - Keep the UI, Improve the Boundary

The original SQLHelper example is excellent for learning CRUD. For a larger application, current Flutter architecture guidance recommends separating the UI from the data source using clear UI and data layers, commonly with a repository/service boundary. This can be taught after the original app works; it does not require replacing the orange screens.

DART • Minimal repository wrapper around the original journal helper

```
class JournalRepository {
  Future<List<Map<String, dynamic>>> loadAll() {
    return SQLHelper.getItems();
  }

  Future<int> create(String title, String description) {
    return SQLHelper.createItem(title, description);
  }

  Future<int> update(int id, String title, String description) {
    return SQLHelper.updateItem(id, title, description);
  }

  Future<void> delete(int id) {
    return SQLHelper.deleteItem(id);
  }
}

// UI asks for intent; repository owns the storage boundary.
final journals = await repository.loadAll();
```

Teaching sequence • Beginner level: call SQLHelper directly so students see CRUD clearly. Engineering level: wrap the same operations behind a repository/service. The screen identity stays the same while testability and maintainability improve.

3.6. Current SQLite Platform and Advanced Notes

- The official Flutter SQLite cookbook still uses the Dog/Fido example and direct sqflite CRUD, so keeping that example is technically current.
- Direct sqflite package support is Android, iOS, and macOS. Linux/Windows can use sqflite_common_ffi; web support follows a different/experimental path.
- Current sqflite supports insert/query/update/delete helpers, transactions, batches, and database version management during open.
- For schema changes after release, keep the version/onUpgrade migration already added in this lecture; editing CREATE TABLE alone does not upgrade an installed database.

4. Preferences with shared_preferences

Preserve the original Save/Retrieve/Delete and restart screens; modernize the API

The original examples remain useful because they teach direct key-value persistence. The technical update is that SharedPreferences is now a legacy API for new work; the current package encourages SharedPreferencesAsync or SharedPreferencesWithCache.



Original SharedPreferences visual retained.

4.1. Current API Choice

API	State model	Read behavior	Use in this lecture
SharedPreferences	Legacy cached singleton	Synchronous getters after initialization	Mention for existing code and migration awareness
SharedPreferencesAsync	No local cache	Every read/write is asynchronous	Default for the updated original examples
SharedPreferencesWithCache	Explicit cache initialized once	Synchronous getters after create()	Optional extension for frequent controlled reads

CRITICAL-DATA WARNING • `shared_preferences` is for simple convenience data. It is not a secure password/token store and does not provide transactional durability guarantees.

4.1.1. Why SharedPreferencesAsync Is the Updated Default in This Lecture

The supported data types did not change: `int`, `double`, `bool`, `String`, and `List<String>`. What changed is the API choice and cache model. `SharedPreferences` is now the legacy API; new code should prefer `SharedPreferencesAsync` or `SharedPreferencesWithCache`.

API	Cache model	When it is useful
SharedPreferences	Legacy local cache	Existing code and migration awareness only.
SharedPreferencesAsync	No local cache; every call is async	Best default for teaching fresh reads and avoiding stale-cache surprises across isolates/engines.
SharedPreferencesWithCache	Explicit local cache	Useful when many synchronous reads are needed and the cache/key policy is controlled.

Android update • With the newer `Async/WithCache` APIs, Android can use `DataStore Preferences`; the package documentation recommends the default `DataStore` option in most cases. This implementation detail does not change the five Dart value types taught in the original lecture.

4.1.2. Migration from Legacy SharedPreferences

If an existing released app already stored values with the legacy `SharedPreferences` API, changing code should not silently orphan old data. The package provides a migration utility for moving legacy values to the newer `Async/WithCache` APIs.

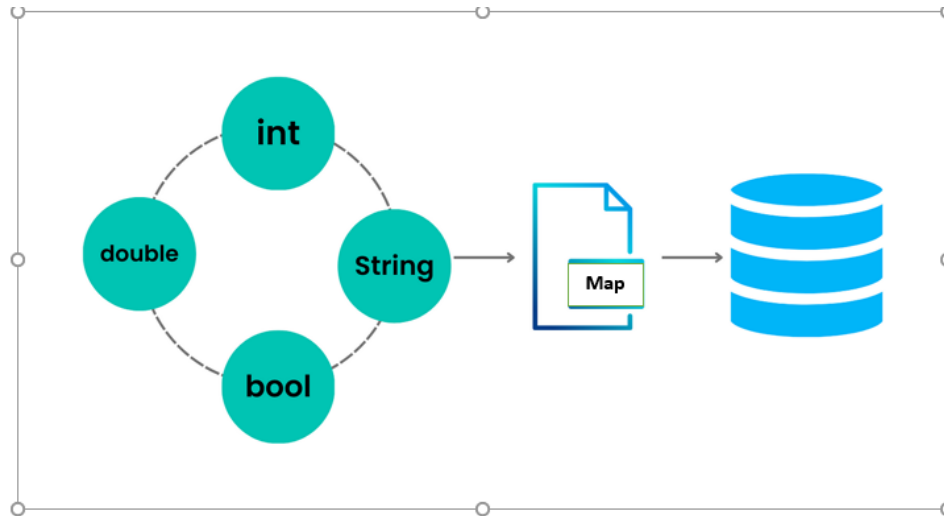
DART • One-time-safe migration utility (advanced note)

```
import 'package:shared_preferences/shared_preferences.dart';
import 'package:shared_preferences/util/legacy_to_async_migration_util.dart';

final legacy = await SharedPreferences.getInstance();
await migrateLegacySharedPreferencesToSharedPreferencesAsyncIfNecessary(
  legacySharedPreferencesInstance: legacy,
  sharedPreferencesOptions: const SharedPreferencesOptions(),
  migrationCompletedKey: 'migrationCompleted',
);
```

Classroom scope • Teach this migration as an advanced maintenance concern after students understand Save/Read/Delete. It is important for real apps, but it should not replace the original per-type learning sequence.

4.2. Supported Types - keep each type as a separate teaching step



Original typed-values diagram retained.

Setter / getter	Dart type	Typical value
setString / getString	String	username, display name, locale
setInt / getInt	int	status, count, selected tab
setDouble / getDouble	double	balance, font scale
setBool / getBool	bool	loggedIn flag, onboarding complete
setStringList / getStringList	List<String>	foodList, tags, recent filters

The five supported value types are still taught separately, as in the original lecture. The engineering update is only the storage API and the way we verify null/default/restart behavior.

Modern setup • For new classroom examples, create one `SharedPreferencesAsync` instance and keep stable keys. Existing released applications that already contain legacy `SharedPreferences` data should use the package migration utility before changing storage backends.

DART • Shared setup used by the separate examples

```
import 'package:shared_preferences/shared_preferences.dart';

final SharedPreferencesAsync prefs = SharedPreferencesAsync();

const String usernameKey = 'username';
const String statusKey = 'status';
const String balanceKey = 'balance';
const String loggedInKey = 'loggedIn';
const String foodListKey = 'foodList';
```

4.2.1. String - Save, Read, Default, and Delete

Use `String` for short textual settings such as a username, display name, language code, or a small label. The getter returns `String?`, so the first-run case must be designed explicitly.

DART • String example - original username idea, modern async API

```
// SAVE
await prefs.setString(usernameKey, 'Eng:Ruqaih Salman');

// READ
final String username =
  await prefs.getString(usernameKey) ?? 'No saved name';
print(username);

// DELETE ONE KEY
await prefs.remove(usernameKey);
```

User action	What the app does
Save	Store the text under the stable key username.

Read	Await getString(). If the key is absent, use an intentional default instead of forcing !.
Delete	remove(usernameKey) removes only this preference; unrelated keys remain.
Verify	Close and relaunch the app, then read the same key to prove restoration.

4.2.2. int - Numeric Whole Values

Use int for whole-number settings such as status codes, launch counters, selected-tab indexes, or small numeric preferences. Avoid using a magic integer when a bool or enum-like model would communicate intent more clearly.

DART • int example - status

```
await prefs.setInt(statusKey, 1);

final int status =
  await prefs.getInt(statusKey) ?? 0;
print(status); // 1 after save, 0 on first run/default

await prefs.remove(statusKey);
```

4.2.3. double - Decimal Settings

Use double for decimal settings such as font scale, volume level, zoom factor, or a non-critical numeric preference. It is not a substitute for a transactional financial database.

DART • double example - balance from the original lecture

```
await prefs.setDouble(balanceKey, 20.5);

final double balance =
  await prefs.getDouble(balanceKey) ?? 0.0;
print(balance); // 20.5

await prefs.remove(balanceKey);
```

Teaching note • The original “balance = 20.5” example is retained to demonstrate the type, but explain that shared_preferences is not appropriate for critical financial records or transaction history.

4.2.4. bool - Flags and On/Off State

Use bool when the meaning is naturally true/false: dark mode enabled, onboarding completed, a simple feature flag, or a non-sensitive “remember preference” choice.

DART • bool example - loggedIn flag retained as a type demonstration

```
await prefs.setBool(loggedInKey, true);

final bool loggedIn =
  await prefs.getBool(loggedInKey) ?? false;
print(loggedIn); // true after save

await prefs.remove(loggedInKey);
```

Security clarification • Keep the bool example only as a flag demonstration. Do not store passwords, authentication tokens, private keys, or other secrets in ordinary shared_preferences.

4.2.5. List<String> - A Small List of Text Values

Use List<String> for a small collection of simple text values such as recent filters, favorite labels, tags, or the original food-list demonstration. It is not intended for a large object graph or queryable records.

DART • List<String> example - original foodList

```
await prefs.setStringList(
  foodListKey,
  <String>['pizza', 'burger', 'sandwich'],
);

final List<String> foodList =
  await prefs.getStringList(foodListKey) ?? <String>[];
print(foodList);

await prefs.remove(foodListKey);
```

4.2.6. Key Existence, remove(), and clear()

The original lecture also taught key existence and deletion separately. Keep that sequence, but make the async behavior explicit and avoid clearing unrelated package keys.

DART • Key-management utilities with SharedPreferencesAsync

```
final bool hasUsername =
  await prefs.containsKey(usernameKey);
print(hasUsername);

// Remove one preference.
await prefs.remove(usernameKey);

// Reset only the keys owned by this lecture feature.
await prefs.clear(
  allowList: <String>{
    usernameKey,
    statusKey,
    balanceKey,
    loggedInKey,
    foodListKey,
  },
);
```

Why use allowList? • A broad clear() can erase preferences owned by other features. A scoped allowList makes Reset behavior deliberate and easier to test.

4.2.7. First Run, Null Values, and Restart Proof

Every getter may return null when the key does not exist. Teach this as a normal state, not an automatic error. After saving, terminate the application process, relaunch it, and read the same key again. This separates persistent storage from widget memory.

User action	What the app does
First launch	Key is absent → getter returns null → UI shows an intentional default/empty state.
Save	Await the setter and show success only after the Future completes.
Terminate app	Widget memory is destroyed; the persisted key should remain.
Relaunch + read	The stored value is read again and the original screen is reconstructed.
Reset	Remove only the intended keys and verify the default state returns.

Identity preserved • The original Save / Retrieve / Delete screen and the “Hello Eng: Ruqaih Salman” restart example remain the practical demonstrations. The detailed type-by-type explanations above restore the teaching depth of the old lecture while using the current asynchronous API.

4.2.8. Visual Recap - All Types + Original Screen Identity

Lecture 9 – Data Persistence in Flutter (Identity Preserved)

Part B – SharedPreferences

Part B – SharedPreferences (Key–Value Storage)

We keep the original examples and screens, but implement them using the modern API (**SharedPreferencesAsync**).



B.1 What is SharedPreferences?

SharedPreferences stores small pieces of data as key–value pairs.
Data is persistent across app restarts but not suitable for large data.
Use it for: user settings, flags, tokens, last-selected item, theme, etc.

Where is it stored?

- Android: SharedPreferences (XML file)
- iOS/macOS: UserDefaults
- Windows/Linux: Local file
- Web: LocalStorage (for supported types)

Important

Do not store sensitive data like passwords, tokens, or private keys in plain SharedPreferences.
Use secure storage instead.

B.2 Overview – Supported Data Types

Original lesson explained each type separately.
We keep the same, but with modern setup.

Supported types:

Type	Example
String	"Ruqaih Salman"
int	42
double	3.14159
bool	true / false
StringList	["eng", "arabic"]

Modern setup (recommended for new apps)

```
import 'package:shared_preferences_async/shared_preferences_async.dart';

class PrefsService {
  static final SharedPreferencesAsync _prefs =
    SharedPreferencesAsync();
  static const String _nameKey = 'username';
  static const String _ageKey = 'age';
  static const String _piKey = 'pi';
  static const String _isDarkKey = 'isDark';
  static const String _langsKey = 'languages';
}
```

Why Async API?

- ✓ No stale cache between isolate/engine boundaries.
- ✓ Works consistently on all platforms.
- ✓ Recommended by official documentation.

[https://pub.dev/documentation/shared_preferences/latest/](https://pub.dev/documentation/shared_preferences/latest/shared_preferences/latest/) (2024+)

B.3 Per-Type Tutorial (Keep the Original Flow – Updated Code)

1) String

```
// Save
await PrefsService._prefs.
  setString(PrefsService._nameKey, 'Ruqaih');

// Read
String? name = await PrefsService._prefs.
  getString(PrefsService._nameKey);

// Delete
await PrefsService._prefs.
  remove(PrefsService._nameKey);
```

Try It
Save your name, read it, then delete it.
Key: username

2) int

```
await PrefsService._prefs.
  setInt(PrefsService._ageKey, 21);

int? age = await PrefsService._prefs.
  getInt(PrefsService._ageKey);

await PrefsService._prefs.
  remove(PrefsService._ageKey);
```

Try It
Save your age as int.
Key: age

3) double

```
await PrefsService._prefs.
  setDouble(PrefsService._piKey, 3.14159);

double? pi = await PrefsService._prefs.
  getDouble(PrefsService._piKey);

await PrefsService._prefs.
  remove(PrefsService._piKey);
```

Try It
Save PI value.
Key: pi

4) bool

```
await PrefsService._prefs.
  setBool(PrefsService._isDarkKey, true);

bool? isDark = await PrefsService._prefs.
  getBool(PrefsService._isDarkKey);

await PrefsService._prefs.
  remove(PrefsService._isDarkKey);
```

Try It
Save a true/false flag.
Key: isDark

5) StringList

```
await PrefsService._prefs.
  setStringList(PrefsService._langsKey,
    ['eng', 'arabic']);

List<String>? langs = await PrefsService._prefs.
  getStringList(PrefsService._langsKey);

await PrefsService._prefs.
  remove(PrefsService._langsKey);
```

Try It
Save a list of languages.
Key: languages

Utilities (Original Concepts Kept)

```
containsKey(String key)
final exists = await PrefsService._prefs.containsKey(key);

remove(String key)
await PrefsService._prefs.remove(key);

clear()
await PrefsService._prefs.clear();

Use carefully: clear() removes EVERYTHING.
```

B.4 Original App Example #1 – Save / Retrieve / Delete (Username)

1 Initial Screen

Open the app.
Type any username.
Then choose an action.

2 After Save

Tap SAVE.
Data is stored permanently.
You will see a success message.

3 After Retrieve

Tap RETRIEVE.
Value is read from storage and displayed.

4 After Delete

Tap DELETE.
The key is removed.
Try RETRIEVE again → nothing.

User Instructions

1. Enter a username.
2. Press SAVE.
3. Close the app completely (swipe away).
4. Re-open the app.
5. Press RETRIEVE → value is still there.
6. Press DELETE → value is removed.
7. Press RETRIEVE again → no value returned.

Key used: username

B.5 Original App Example #2 – Hello Eng: Ruqaih Salman (Restart Demonstration)

1 First Run

Type your name.
Tap Save Name.

2 After Save

Name is saved.
Message is shown.

3 Close App

Close the app fully (not just back).

4 Re-open App

Name is still here!
Persistence verified.

What's Happening?

- When you save the name, it is stored in persistent storage.
- Closing the app destroys memory but not storage.
- When you open the app again, the saved value is loaded and displayed.

Key used: savedName

MODERN TEACHING INFOGRAPHIC • Visual recap of the five SharedPreferences types and the two original app demonstrations. It summarizes the section after each type has been taught separately; it does not replace the detailed examples.

How to use this image • Use it at the end of the SharedPreferences explanation as a synthesis page: students first learn String/int/double/bool/List<String> one by one, then use the infographic to connect all types to Save/Read/Delete and restart verification.

DART • Optional SharedPreferencesWithCache example

```
final prefsWithCache = await SharedPreferencesWithCache.create(
  cacheOptions: const SharedPreferencesWithCacheOptions(
    allowList: <String>{'username', 'darkMode'},
  ),
);

await prefsWithCache.setBool('darkMode', true);
final darkMode = prefsWithCache.getBool('darkMode') ?? false;
```

4.2.9. Console-First Complete Typed Verification

After teaching String, int, double, bool, and List<String> separately, run one compact verification program that proves defaults, typed writes, reads, key existence, one-key removal, and scoped reset before connecting the logic to the original blue screens.

Before you run	Decision
Entry file	lib/preferences_console_demo.dart
Package	shared_preferences
API	SharedPreferencesAsync for the updated classroom implementation
Run command	flutter run -t lib/preferences_console_demo.dart

DART • preferences_console_demo.dart - complete typed proof

```
import 'package:flutter/widgets.dart';
import 'package:shared_preferences/shared_preferences.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = SharedPreferencesAsync();

  const usernameKey = 'username';
  const statusKey = 'status';
  const balanceKey = 'balance';
  const loggedInKey = 'loggedIn';
  const foodListKey = 'foodList';

  // Start from an intentional first-run state for lecture reproducibility.
  await prefs.clear(
    allowList: <String>{
      usernameKey,
      statusKey,
      balanceKey,
      loggedInKey,
      foodListKey,
    },
  );

  print('[DEFAULT] username=${await prefs.getString(usernameKey)}');
  print('[DEFAULT] status=${await prefs.getInt(statusKey) ?? 0}');

  await prefs.setString(usernameKey, 'Eng:Ruqaih Salman');
  await prefs.setInt(statusKey, 1);
  await prefs.setDouble(balanceKey, 20.5);
  await prefs.setBool(loggedInKey, true);
  await prefs.setStringList(
    foodListKey,
    <String>['pizza', 'burger', 'sandwich'],
  );

  print('[STRING] ${await prefs.getString(usernameKey)}');
  print('[INT] ${await prefs.getInt(statusKey)}');
  print('[DOUBLE] ${await prefs.getDouble(balanceKey)}');
  print('[BOOL] ${await prefs.getBool(loggedInKey)}');
  print('[LIST] ${await prefs.getStringList(foodListKey)}');
  print('[EXISTS] username=${await prefs.containsKey(usernameKey)}');

  await prefs.remove(usernameKey);
  print('[REMOVE] username=${await prefs.getString(usernameKey)}');

  await prefs.clear(
```

```

allowList: <String>{
  statusKey,
  balanceKey,
  loggedInKey,
  foodListKey,
},
);
print('[RESET] status=${await prefs.getInt(statusKey) ?? 0}');
}

```

EXPECTED DEBUG CONSOLE OUTPUT

```

$ flutter run -t lib/preferences_console_demo.dart
[DEFAULT] username=null
[DEFAULT] status=0
[STRING] Eng:Ruqaih Salman
[INT] 1
[DOUBLE] 20.5
[BOOL] true
[LIST] [pizza, burger, sandwich]
[EXISTS] username=true
[REMOVE] username=null
[RESET] status=0

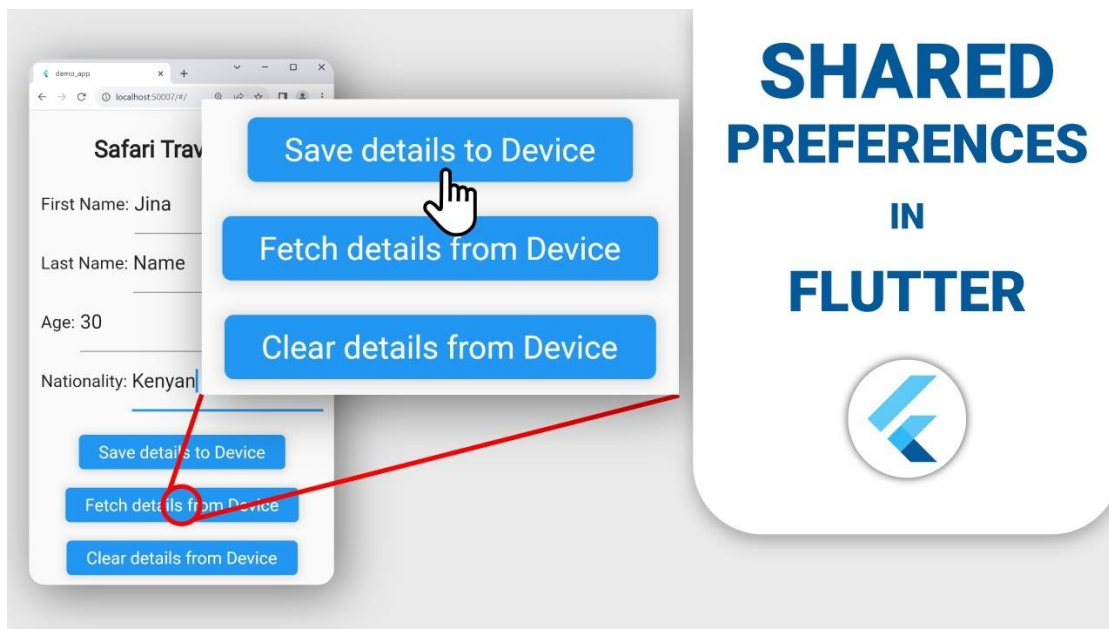
```

Output group	Interpretation
DEFAULT	A missing key is a valid first-run condition; the UI must choose a deliberate default.
STRING/INT/DOUBLE/BOOL/LIST	Each supported Dart type is written and read back with its own typed API.
EXISTS	containsKey() distinguishes absence from a value that merely looks empty.
REMOVE	remove(key) deletes one preference without touching unrelated keys.
RESET	clear(allowList: ...) resets only the feature-owned keys.

TEACHING TRANSITION • Now reuse the same username key and Save/Read/Delete behavior in the original SharedPreferences Roq App. The blue screen identity stays; only the storage implementation is modernized to the current asynchronous API.

4.3. Original Example 1: Save / Retrieve / Delete

Same screen identity, updated asynchronous storage boundary



Original explanatory image retained: save, fetch, and clear details from the device.

DART • SharedPreferences Roq App - updated storage methods

```

class _MyHomePageState extends State<MyHomePage> {
  static const _usernameKey = 'username';
  final SharedPreferencesAsync _prefs = SharedPreferencesAsync();
  final TextEditingController controller = TextEditingController();
  String name = '';

```

```

bool busy = false;

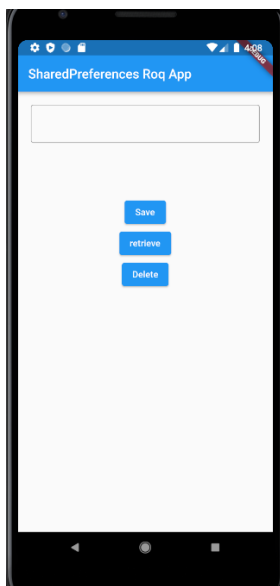
Future<void> save() async {
  setState(() => busy = true);
  try {
    await _prefs.setString(_usernameKey, controller.text.trim());
  } finally {
    if (mounted) setState(() => busy = false);
  }
}

Future<void> retrieve() async {
  final value = await _prefs.getString(_usernameKey) ?? "";
  if (mounted) setState(() => name = value);
}

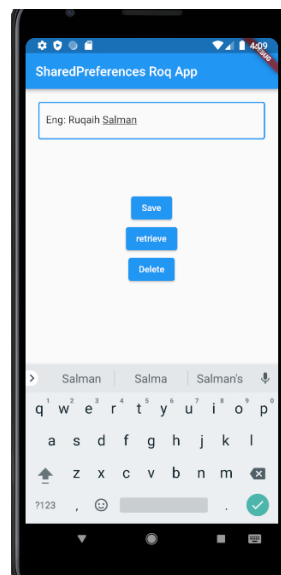
Future<void> delete() async {
  await _prefs.remove(_usernameKey);
  if (!mounted) return;
  setState(() => name = "");
}

@override
void dispose() {
  controller.dispose();
  super.dispose();
}
}

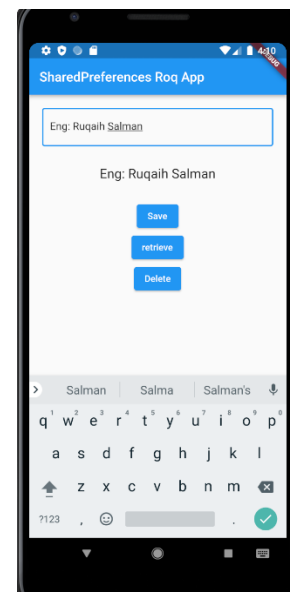
```



Initial



Save text



Retrieve

Figure 5. Original Save/Retrieve/Delete screen sequence retained.

4.4. Original Example 2: Restore the Saved Name After Restart

The restart proof remains part of the lecture identity

DART • Restore name on startup with current Async API

```

class _HomePageState extends State<HomePage> {
  static const _nameKey = 'name';
  final SharedPreferencesAsync _prefs = SharedPreferencesAsync();
  final TextEditingController _nameController = TextEditingController();
  String savedName = "";
  bool loading = true;

  @override
  void initState() {
    super.initState();
    _retrieveName();
  }

  Future<void> _retrieveName() async {
    final value = await _prefs.getString(_nameKey) ?? "";
    if (!mounted) return;
    setState(() {
      savedName = value;
      loading = false;
    });
  }
}

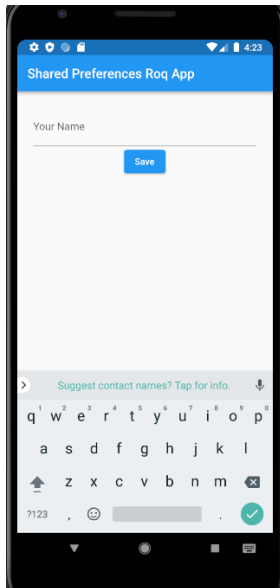
```

```

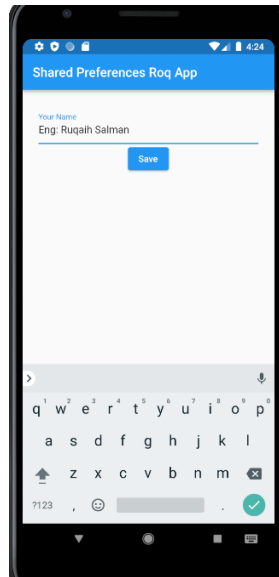
Future<void> _saveName() async {
  final value = _nameController.text.trim();
  await _prefs.setString(_nameKey, value);
  if (mounted) setState(() => savedName = value);
}

Future<void> _clearName() async {
  await _prefs.remove(_nameKey);
  if (mounted) setState(() => savedName = "");
}
}

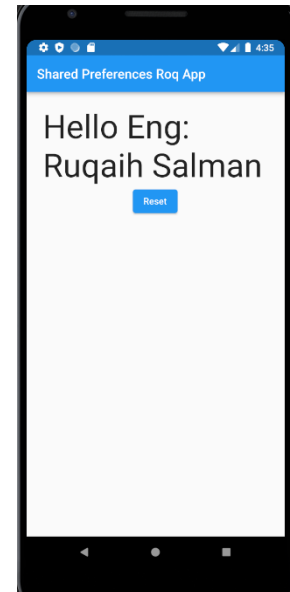
```



Enter name



Save



Relaunch → restored

Figure 6. Original restart/restoration sequence retained, including “Hello Eng: Ruqaih Salman”.

MIGRATION NOTE • Existing apps that already use the legacy SharedPreferences API can migrate deliberately. The lecture does not pretend the old API never existed; it teaches the modern choice while preserving the original screen behavior.

5. File Storage with path_provider and dart:io

Keep the counter and Textfile Roq App; add JSON as an engineering extension

The original file examples are still aligned with Flutter's current recipe: find a path, create a File, write, then read. The update is safer parsing, platform notes, lifecycle handling, path joining, and explicit malformed-file behavior.

5.1. Project Setup and Directory Choice

TERMINAL • Add file-system location + portable path joining

```
flutter pub add path_provider path
```

Directory API	Use	Lifetime / caution
getTemporaryDirectory()	Disposable cache and recomputable downloads	The OS may clear it.
getApplicationSupportDirectory()	Internal support files	App-private; platform-specific.
getApplicationDocumentsDirectory()	Persistent app documents/user-generated content	Common teaching choice; normally removed with the app.
getDownloadsDirectory()	Files intended for user export	Availability/behavior varies by platform.

WEB BOUNDARY • The standard Flutter cookbook path_provider + dart:io read/write-files recipe applies to mobile/desktop, not Flutter web as written.

5.2. Original Official Counter Recipe - Retained and Corrected

DART • CounterStorage - current parsing behavior

```
class CounterStorage {
  Future<String> get _localPath async {
    final directory = await getApplicationDocumentsDirectory();
    return directory.path;
  }

  Future<File> get _localFile async {
    final path = await _localPath;
    return File('$path/counter.txt');
  }

  Future<File> writeCounter(int counter) async {
    final file = await _localFile;
    return file.writeAsString('$counter');
  }

  Future<int> readCounter() async {
    try {
      final file = await _localFile;
      final contents = await file.readAsString();
      return int.parse(contents);
    } catch (_) {
      return 0;
    }
  }
}
```

CORRECTION FROM THE OLD SLIDE • The earlier lecture showed a Future<int> method returning the raw String contents. The current version parses the file text with int.parse(contents), matching the method's declared return type.

5.2.1. Console-First File Boundary Proof

Before using a TextField, prove that path_provider locates a writable app directory and that dart:io can create, read, overwrite, and delete a real file. This makes the difference between widget memory and file state observable.

Before you run	Decision
Entry file	lib/file_console_demo.dart
Packages	path_provider + path
File	counter.txt in Application Documents
Run command	flutter run -t lib/file_console_demo.dart

DART • file_console_demo.dart - complete file-system proof

```
import 'dart:io';

import 'package:flutter/widgets.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
```

```

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final directory = await getApplicationDocumentsDirectory();
  final file = File(p.join(directory.path, 'counter.txt'));

  if (await file.exists()) {
    await file.delete();
  }

  print('[ABSENT] exists=${await file.exists()}');

  await file.writeAsString('7', flush: true);
  print('[WRITE] raw=${await file.readAsString()}');

  final counter = int.parse(await file.readAsString());
  print('[READ] counter=$counter');

  await file.writeAsString('${counter + 1}', flush: true);
  print('[OVERWRITE] counter=${int.parse(await file.readAsString())}');

  await file.delete();
  print('[DELETE] exists=${await file.exists()}');
}

```

EXPECTED DEBUG CONSOLE OUTPUT

```

$ flutter run -t lib/file_console_demo.dart
[ABSENT] exists=false
[WRITE] raw=7
[READ] counter=7
[OVERWRITE] counter=8
[DELETE] exists=false

```

Console marker	What it proves
[ABSENT]	The file is not present before the first write; this is a valid empty/first-run state.
[WRITE]	writeAsString() created the file and persisted the text representation.
[READ]	The stored String can be read and deliberately parsed back to int.
[OVERWRITE]	A later write changes the file itself, not merely an in-memory variable.
[DELETE]	The physical file was removed and an existence check confirms it.

TEACHING TRANSITION • Now connect the same file boundary to the original Textfile Roq App: TextField input is temporary widget state; Save writes data.txt; Read reconstructs visible text from the file.

Use the original three-screen sequence as the UI proof, then follow it with the JSON extension to show how a simple text file evolves into a validated structured format.

5.3. Original UI Example: Textfile Roq App

Same name-entry screens; safer file and lifecycle handling

DART • Textfile Roq App - updated implementation

```

import 'dart:convert';
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

class _HomePageState extends State<HomePage> {
  final _textController = TextEditingController();
  String? _content;
  Object? _error;
  bool _busy = false;

  Future<File> _dataFile() async {
    final directory = await getApplicationDocumentsDirectory();
    return File(p.join(directory.path, 'data.txt'));
  }

  Future<void> _writeData_funroq() async {
    setState(() => _busy = true);
    try {
      final file = await _dataFile();
      await file.writeAsString(_textController.text, encoding: utf8);
      _textController.clear();
    } catch (error) {

```

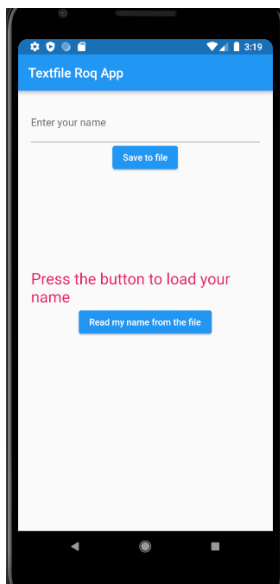
```

    if (mounted) setState(() => _error = error);
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}

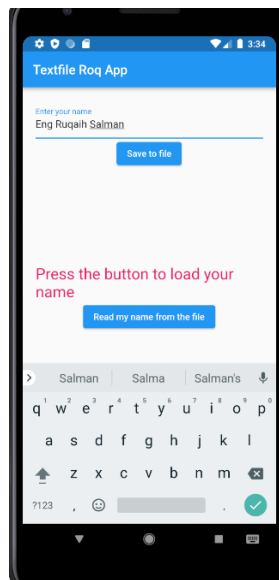
Future<void> _readData_funroq() async {
  setState(() => _busy = true);
  try {
    final file = await _dataFile();
    final data = await file.exists()
      ? await file.readAsString(encoding: utf8)
      : "";
    if (!mounted) return;
    setState(() {
      _content = data;
      _error = null;
    });
  } catch (error) {
    if (mounted) setState(() => _error = error);
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}

@override
void dispose() {
  _textController.dispose();
  super.dispose();
}
}

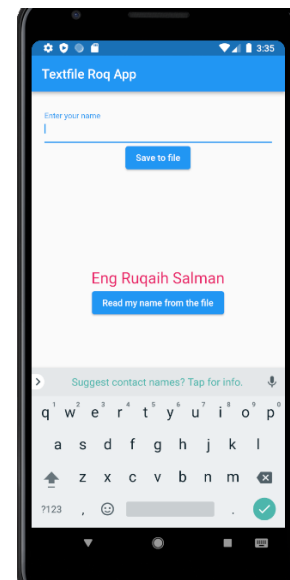
```



Initial



Enter + Save



Read from file

Figure 7. Original Textfile Roq App output sequence retained.

5.3.1. Current path_provider and Version Policy

The old lecture pinned package versions such as path_provider ^1.6.27. For a current lecture, prefer flutter pub add so Pub resolves versions compatible with the installed Flutter SDK. Current path_provider supports Android, iOS, Linux, macOS, and Windows, but the standard dart:io file recipe still does not apply to Flutter web.

TERMINAL • Resolve compatible current packages

```
flutter pub add path_provider path
```

Current package note • Recent path_provider releases also document more directory APIs and platform differences. Keep the original Documents-directory example because it is still valid, but teach directory choice as part of data lifetime rather than as a fixed universal path.

5.4. Engineering Extension: Structured JSON

Add structure after the original text-file concept, not instead of it

DART • local_note.dart - explicit JSON schema and validation

```

class LocalNote {
  const LocalNote({required this.text, required this.savedAt});

```

```

final String text;
final DateTime savedAt;

Map<String, Object?> toJson() => <String, Object?>{
  'text': text,
  'savedAt': savedAt.toIso8601String(),
};

factory LocalNote.fromJson(Map<String, dynamic> json) {
  final text = json['text'];
  final savedAt = json['savedAt'];
  if (text is! String || savedAt is! String) {
    throw const FormatException('Invalid local-note schema');
  }
  return LocalNote(
    text: text,
    savedAt: DateTime.parse(savedAt),
  );
}

```

DART • JSON file service - missing file is empty, malformed JSON is error

```

Future<LocalNote?> readNote(File file) async {
  if (!await file.exists()) return null;

  final raw = await file.readAsString();
  final decoded = jsonDecode(raw);
  if (decoded is! Map<String, dynamic>) {
    throw const FormatException('Expected a JSON object');
  }
  return LocalNote.fromJson(decoded);
}

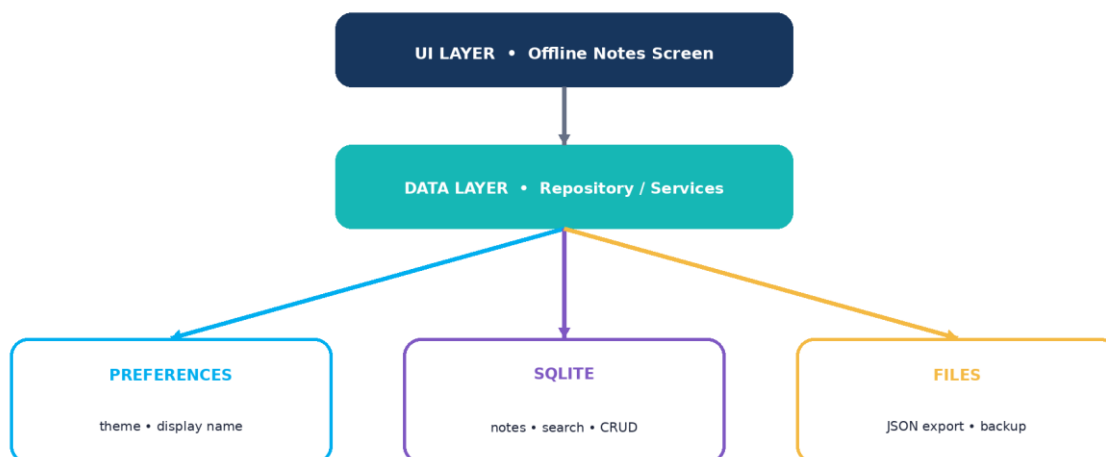
```

Condition	Service result	Recommended UI
File absent	null	First-run / empty state
Valid JSON	Typed LocalNote	Populate field and show restored state
Malformed JSON/schema	FormatException	Error + reset/recovery path
I/O failure	FileSystemException / other error	Do not claim success; offer retry/details

6. Integrated Architecture and Offline-First Thinking

One authoritative home per datum, one data boundary for the UI

ONE FEATURE, THREE PURPOSE-BUILT STORES



The UI asks for intent; services own storage details; each datum has one authoritative home.

Figure 8. Preferences own settings, SQLite owns queryable records, and files own export/backup documents.

Datum	Authoritative home	Why
Display name / simple flag	Preferences	Small direct lookup; no query model needed.
Journals / records	SQLite	Many rows, order/filter, CRUD, migration.
Exported TXT/JSON	File	Portable or explicit document representation.

Datum	Authoritative home	Why
Current TextField text	Widget/controller state	Ephemeral edit buffer until Save.
Remote synchronized copy	Server + repository policy	Not automatically solved by local persistence.

6.1. Offline-First Is a Synchronization Policy

- Local-only: device storage is the source of truth and there is no remote synchronization.
- Local fallback: try remote data; use local copy when the network is unavailable.
- Offline-first write: save locally, mark pending, then synchronize later.
- Conflict policy: decide what happens if local and remote copies both change.
- Repository boundary: UI asks for intent; repository coordinates local/remote sources.

6.2. Security, Privacy, and Data Minimization

Risk	Why it matters	Direct control
Secrets in ordinary storage	Local files/preferences/database can be inspected on a compromised device or backup.	Use secure-storage/encryption design; avoid unnecessary secrets.
Excess retention	Old personal data remains after the feature no longer needs it.	Define delete/reset and retention behavior.
Sensitive logs	Debug output can expose values.	Do not print tokens/passwords/private records.
Unscoped clear/delete	One feature may erase unrelated data.	Use stable keys, allowLists, and explicit targets.
Unvalidated import	User-controlled file can be malformed/hostile.	Validate type, size, schema, and range.

6.3. Integrated UI State Checklist

State	What the user sees	What code owns
Loading	Progress indicator or skeleton	One in-flight Future and disabled duplicate action
Empty	Purposeful first-run message	null / absent file / zero rows mapped to a valid state
Data	Restored settings or records	Validated deserialization / query result
Saving	Disabled action or progress	Awaited write and error boundary
Success	Truthful confirmation	Only after the storage Future completes successfully
Error	Explanation + retry/reset path	Exception retained long enough for diagnosis

DUAL-EVIDENCE RULE • For SQLite, Preferences, and Files, every main example must have two aligned artifacts: (1) ordered Debug Console output proving the storage contract and (2) matching Flutter screen states proving that the UI renders the stored result. For persistence claims, add save → terminate → relaunch → read whenever the mechanism should survive process death.

7. Debugging and Testing

Trace lifetime, path, representation, and visible state

7.1. Common Failures and Direct Corrections

Symptom	Likely cause	Direct correction
Value resets after restart	Only widget memory changed; write not called/awaited	Instrument the storage call; relaunch and read from storage.
Database table not found	Wrong path or onCreate not run	Verify path/version; recreate only the development DB if appropriate.
New column missing	CREATE TABLE edited but installed DB stayed on old version	Increase version and implement/test onUpgrade.
Wrong row updated/deleted	Unsafe/missing WHERE	Use where: id = ? with whereArgs.
Preference getter returns null	First run, wrong key, or spelling changed	Use key constants and an intentional default.
Preference type exception	Same key written with a different type	Keep one type per key; migrate/remove incompatible data.
Missing file	Read before first write or wrong path/name	Check exists(); treat absence as a valid empty state.
FormatException	Malformed JSON/schema mismatch	Validate structure and provide recovery/reset.
setState after dispose	Async result completed after page left tree	Check mounted after await.
UI says saved but data absent	Success shown before awaited write completed	Show success only after successful completion.

7.2. Diagnostic Sequence

1. Confirm the user action produces the expected value before serialization.

2. Confirm the helper/service method is called once and its Future is awaited.
3. Compare write and read identifiers: key, table/database path, or file path.
4. Inspect raw stored representation and compare its type/schema with the model conversion.
5. Restart the process and invoke the read path independently of original widget memory.
6. Verify null/empty/error branches and mounted checks.
7. Only then investigate package/platform-specific behavior.

7.3. Test the Boundary, Not the Plugin Details

Test level	Exercise	Evidence
Model test	toMap/fromMap or toJson/fromJson round trip	Values preserved; invalid schema rejected.
Service/helper unit test	Fake/in-memory boundary	Read/write/remove/default behavior without UI.
Migration test	Open old schema with new version	Rows survive and new column/default exists.
Widget test	Inject loading/data/error service states	Visible branches and disabled actions are correct.
Integration test	Save, terminate/recreate, read on a supported target	Plugin + store + UI work together.

EVIDENCE RULE • For each practical store, capture both the ordered Debug Console evidence and the matching original/updated Flutter screen sequence. A same-run screenshot alone does not prove persistence.

8. Guided Practice Lab

Identity-preserving Study Vault: reuse the lecture examples instead of introducing unrelated replacement screens

The lab combines the same examples students already saw: Dog/Fido for basic SQLite proof, the original Journal app for CRUD UI, the original name/preferences screen for settings, and the original text-file screen before a JSON extension.

Checkpoint	Task	Acceptance evidence
1	Run Dog/Fido console CRUD exactly once from a Flutter entry point.	Insert → query → update → query → delete; console evidence.
2	Run the original Sqflite Roq App and create/update/delete one journal.	Original orange UI screens + queried results.
3	Upgrade the journal DB to version 2 with pinned default 0.	Old rows survive migration.
4	Run SharedPreferences Roq App with Save/Retrieve/Delete using Async API.	Original blue screens + values read back.
5	Terminate and relaunch the saved-name example.	Hello screen restores saved name from a fresh process.
6	Run Textfile Roq App: Save then Read.	Original three screen states + actual data.txt persistence.
7	Add JSON export of one journal.	Validated save/read and malformed-file error path.
8	Write one paragraph explaining why each datum uses its chosen store.	Data ownership reasoning + security boundary.

8.1. Marking Rubric

Criterion	Evidence	Weight
Storage selection	Each datum has a justified authoritative store	15%
SQLite correctness	Dog console proof + original Journal CRUD + migration + restart	25%
Preferences correctness	Typed Async proof + original Save/Retrieve/Delete + restart	15%
File correctness	Counter/text file + original screen sequence + JSON extension	15%
Async UI	Loading/empty/error/saving and mounted safety	15%
Architecture/testing	Helper/service boundary + at least two tests	10%
Code quality	Naming, disposal, comments, concise explanation	5%

9. Knowledge Check and Assessment

Explain, predict, select, correct, and build

9.1. Multiple Choice

1. Which store best fits 2,000 journal rows that must be filtered and ordered? A) widget field B) SharedPreferencesAsync C) SQLite D) temporary file
2. Which shared_preferences API is the recommended uncached asynchronous option for new work? A) SharedPreferencesAsync B) only SharedPreferences.getInstance() C) File D) rawQuery
3. What is the safest way to target one SQLite row by an external id? A) interpolate it B) where: 'id = ?' + whereArgs C) delete all D) use a preference key
4. Which directory is appropriate for disposable recomputable cache? A) application documents B) temporary directory C) database path only D) widget build directory
5. What proves persistence most directly? A) Text changes B) SnackBar appears C) setState runs D) data returns after process restart

9.2. True / False - Correct the False Statements

1. SharedPreferences is an encrypted store suitable for passwords.
2. A State field can survive rebuilds while mounted, but process memory disappears when the app process is killed.
3. Editing CREATE TABLE alone upgrades an already-installed SQLite database.
4. A missing preference key can represent a normal first-run state.
5. whereArgs keeps external values separate from SQL structure.
6. The standard path_provider + dart:io cookbook recipe works unchanged on Flutter web.
7. A transaction can provide all-or-nothing behavior for dependent database operations.
8. Success feedback should normally appear only after an awaited storage write succeeds.

9.3. Code Reading and Correction

DART • Question A - identify two boundary problems

```
Future<void> removeJournal(int id) async {
  final db = await SQLHelper.db();
  await db.delete('items', where: 'id = $id');
  setState(() => message = 'Deleted');
}
```

Question B. Explain why repeatedly creating a storage Future inside build() can cause duplicate reads, and list the missing loading/error/empty UI branches.

DART • Question C - why will this read not restore the write?

```
final prefs = SharedPreferencesAsync();
final dark = await prefs.getBool('dark-mode') ?? false;
await prefs.setBool('dark_mode', true);
```

10. Answer Key, Quick Reference, and Official Sources

Concise model answers plus a verified technical baseline

10.1. Answer Key

Assessment	Answer / model correction
MCQ 1–5	1 C • 2 A • 3 B • 4 B • 5 D
T/F 1	False - ordinary preferences are not a secure password store.
T/F 2	True.
T/F 3	False - increase the database version and implement/test onUpgrade.
T/F 4	True - absence may be a valid first-run state.
T/F 5	True.
T/F 6	False - standard dart:io file recipe is mobile/desktop, not web.
T/F 7	True.
T/F 8	True unless the app intentionally implements an optimistic UI strategy.
Code A	Use where: 'id = ?' with whereArgs: [id]. Keep UI state safe across await with mounted checks.
Code B	Create/cache the Future outside repeated build work and render waiting/error/empty/data branches.
Code C	The keys differ: dark-mode vs dark_mode. Declare a single key constant.

10.2. Quick Reference

Need	API / pattern	Remember
Add packages	flutter pub add <package>	Resolve versions with the current Flutter SDK instead of copying stale tutorial pins.
SQLite path	join(await getDatabasesPath(), name)	One database path; explicit version.
SQLite CRUD	insert / query / update / delete	Map data consistently; parameterize WHERE.
Schema change	version + onUpgrade	Test a real old→new migration.
Atomic database work	db.transaction((txn) async { ... })	Use txn inside the transaction callback.
Preference read	await prefs.getString(key) / getBool(key)	Getter may return null; define a first-run default.
Preference write	await prefs.setString / setBool / ...	Simple non-critical typed settings.
Scoped preference reset	clear(allowList: {...})	Avoid erasing unrelated keys.
File path	getApplicationDocumentsDirectory()	Choose the directory from the required lifetime.
File read/write	readAsString / writeAsString	Check existence and validate parsed schema.
Persistence proof	save → terminate → relaunch → read	Same-run screenshots are insufficient.

10.3. Official Sources and Verification Baseline

TECHNICAL BASELINE • Reviewed against official Flutter/Dart package documentation on 11 August 2026. Installation commands intentionally use flutter pub add rather than pinned versions.

- Flutter Cookbook - Persist data with SQLite: <https://docs.flutter.dev/cookbook/persistence/sqlite>
- Flutter Cookbook - Store key-value data on disk: <https://docs.flutter.dev/cookbook/persistence/key-value>
- Flutter Cookbook - Read and write files: <https://docs.flutter.dev/cookbook/persistence/reading-writing-files>
- sqflite package: <https://pub.dev/packages/sqflite>
- shared_preferences package/API: https://pub.dev/packages/shared_preferences and https://pub.dev/documentation/shared_preferences/latest/
- path_provider package: https://pub.dev/packages/path_provider
- Dart File API: <https://api.dart.dev/dart-io/File-class.html>
- Flutter architecture - Offline-first support: <https://docs.flutter.dev/app-architecture/design-patterns/offline-first>

VERSION POLICY • Before each academic year, resolve package versions with the current Flutter SDK and review breaking changes and platform minimums. Keep the teaching example when it remains valid; update the API beneath it when necessary.

10.3.1. Visual Engineering Handout

Lecture 9 - Data Persistence in Flutter

Part 1: SQLite - Example 2 (UI)

Example App: Recipe Flutter Light (Journal / Items)

A simple diary/journal app that stores items (title + description) in an SQLite database using sqflite.

APP OVERVIEW

You will build a simple CRUD (Create, Read, Update, Delete) application with the following features:

- Add a new note (title + description)
- View all notes
- Update existing note
- Delete a note

WORKFLOW (USER JOURNEY)

- Tap the (Floating Action Button) to add a new note.
- Fill the Bottom Sheet and tap Save.
- The new note appears in the list.
- Tap an item to edit it.
- Modify the fields and tap Save.
- Long press an item (or tap) to delete it and confirm.
- The list refreshes automatically.

KEY COMPONENTS

- SQLite database (table: items)
- sqflite package
- Bottom Sheet for Create / Update
- ListView to display items
- Dialog for Delete confirmation

TABLE: items

Column	Type	Description
id	INTEGER	PRIMARY KEY AUTOCREMENT
title	TEXT	Note title
description	TEXT	Note description

TIP

All operations are performed on the local device and persist even after the app is closed.

BEHIND THE SCENES (WHAT HAPPENS)

CREATE

- Insert new row into items table
- Get the new id
- Refresh list from database

READ

- Query all rows from items
- Map rows to item objects
- Display in ListView

UPDATE

- Update the row where id = ?
- Use new values (title, description)
- Refresh list

DELETE

- Delete the row where id = ?
- Refresh list

Part 2: SharedPreferences - All Types (Separated)

SharedPreferences is used to store simple key-value pairs (small data) on the device.

API (Recommended for new projects)

- Use `SharedPreferencesAsync` (no cache issues, safe for all platforms).

```
import 'package:shared_preferences_async/shared_preferences_async.dart';
```

GET INSTANCE

```
final prefs = SharedPreferencesAsync();
```

TIP

Every operation is async, so use await.

1) STRING (String)

Save

```
await prefs.setString('username', 'Ruqaih Hussein');
```

Read

```
final name = await prefs.getString('username');
print(name); // Ruqaih Hussein
```

Delete

```
await prefs.remove('username');
```

Check Exists

```
final exists = await prefs.containsKey('username');
```

2) INT (Integer)

Save

```
await prefs.setInt('age', 20);
```

Read

```
final age = await prefs.getInt('age');
```

Delete

```
await prefs.remove('age');
```

3) DOUBLE (Double / Float)

Save

```
await prefs.setDouble('height', 165.5);
```

Read

```
final h = await prefs.getDouble('height');
```

Delete

```
await prefs.remove('height');
```

4) BOOL (Boolean)

Save

```
await prefs.setBool('isLoggedIn', true);
```

Read

```
final isLoggedIn = await prefs.getBool('isLoggedIn');
```

Delete

```
await prefs.remove('isLoggedIn');
```

5) STRING LIST (List<String>)

Save

```
await prefs.setStringList('fruits', ['Apple', 'Banana', 'Orange']);
```

Read

```
final fruits = await prefs.getStringList('fruits');
print(fruits); // Apple, Banana, Orange
```

Delete

```
await prefs.remove('fruits');
```

COMMON UTILITIES

Check if a key exists

```
final exists = await prefs.containsKey('key');
```

Remove a key

```
await prefs.remove('key');
```

Clear all preferences

```
await prefs.clear();
```

BEST PRACTICES

- Use for small data only (settings, flags, tokens, usernames, theme...).
- Do not store large objects (use SQLite or File instead).
- Do not store sensitive data (passwords, private keys) without encryption.
- Always handle null values (first run).
- Use meaningful keys and keep them consistent.

COMMON USE CASES

- Remember username
- Save user settings (theme, font size)
- Login session token (small string)
- App first-run flag
- Feature enable/disable flags

MODERN TEACHING INFOGRAPHIC • One-page engineering handout combining the original SQLite identity with the current SharedPreferences guidance. Best used for end-of-lecture review or as a student handout.

Professional placement strategy • Original screenshots stay beside the code they prove. Reconstructed screens stay beside the missing operation they explain. Dense infographics belong at section-end or lecture-end as synthesis/recap pages, where they support memory without replacing executable evidence.

Current verification baseline - checked 12 August 2026 • Flutter documentation pages used for this revision reflect Flutter 3.44.7 (docs updated May 2026). Current package baselines checked during revision: shared_preferences 2.5.5, sqflite 2.4.3, path_provider 2.1.6. Lecture commands intentionally use flutter pub add rather than hard-coded dependency pins so the material remains maintainable.

FINAL METHOD ADOPTED • Original example identity + console-first complete proof + expected output + full Flutter UI + original/reconstructed screen sequence + detailed execution instructions + restart/failure evidence + current engineering update. New guidance is layered around the old example; it does not erase it.

10.4. Identity Preservation Summary

Original lecture element	Final treatment
Dog / Fido SQLite CRUD	Retained; aligned with the current official Flutter cookbook and cleaned for typing/safe selection.
Sqflite Roq App / orange journal screens	Retained; helper and lifecycle rules modernized without changing visible teaching identity.
SharedPreferences Save / Retrieve / Delete screens	Retained; storage calls moved to SharedPreferencesAsync for new-code guidance.
Hello Eng: Ruqaih Salman restart screen	Retained as the explicit process-restart persistence proof.
counter.txt recipe	Retained and corrected so readCounter() parses String → int.
Textfile Roq App / blue screens	Retained; file path, existence, error, mounted, and dispose handling improved.
JSON storage	Added after the original text-file example as an extension, not as a replacement.
Engineering structure	Mental model, selection, migration, security, testing, architecture, lab, and assessment retained from the re-engineered edition.
Original vs reconstructed screen provenance	Added explicit labels so generated/reconstructed visuals are never presented as historical emulator output.
Material 3 visual differences	Explained as a current Flutter UI change; legacy look can be demonstrated temporarily without changing persistence logic.
Modern architecture guidance	Added repository/service extension after the original SQLHelper teaching path; original UI remains intact.
SharedPreferences migration/cache model	Added Async/WithCache differences, Android DataStore note, and legacy migration example while retaining per-type instruction.
Generated teaching infographics	Placed as section-end synthesis/handout visuals, clearly labeled as teaching graphics rather than runtime evidence.